



AGRUPAMENTO DE ESCOLAS
EDUARDO GAGEIRO
Escola Secundária de Sacavém

**PROFESSIONAL COURSE IN COMPUTER SYSTEMS
MANAGEMENT AND PROGRAMMING**

PROFESSIONAL APTITUDE PROJECT

Project Report
Academic Year 2025/2026

ARMIAC - Intelligent Arm



Quévin Leonardo Aguiar Tavares

2025/2026

Supervising Teachers: Cláudia Barata & Paula Alcobia

Contents

- Acknowledgements
- 1 - Introduction
- 1.1 - Objective and real-world application in education
- 2 - Technologies Involved
- 2.1 - Software Used
- 2.2 - Devices Used
- 2.3 - Programming Languages
- 2.4 - Technologies and Communication
- 2.5 - Electronic Components Used
- 2.6 - 3D Printing and Assembly Materials
- 3 - Project Implementation
- 4 - Project Development
- 4.1 - Timeline and Development Phases
- 4.2 - Prototype Development
- 4.2.1 - Creation of the 3D Models
- 4.2.2 - Preparation of the 3D Models
- 4.2.3 - 3D Printing of the Parts
- 4.2.4 - Assembly of the Fingers and Joints
- 4.2.5 - Assembly of the Hand and Wrist
- 4.2.6 - Installation of the Forearm Servomotors
- 4.2.7 - Cables and Extensions
- 4.2.8 - Fitting the Hand and Positioning the Lines
- 4.2.9 - Creation of the ARMIAC Base
- 4.2.10 - Electrical Tests and Multimeter Measurements
- 4.2.11 - Connections Between the ESP32 and PCA9685
- 4.2.12 - Power Supply and Initial Tests
- 4.3 - ESP32 Programming (Arduino IDE)
- 4.4 - Flutter Application Programming
- 4.4.1 - Creating the Flutter Project
- 4.4.2 - Dependency and Resource Management (pubspec.yaml)
- 4.4.3 - Flutter Project Organization (Folder Structure)
- 4.4.4 - Generating the Application APK
- 4.5 - Project Visual Identity (ARMIAC Logo)
- 4.6 - Difficulties Encountered During Development
- 5 - Conclusion
- 5.1 - Future Improvements
- 6 - Web References
- 7 - Appendices
- 7.1 - Source Code
- 7.2 - ARMIAC Application APK
- 7.3 - YouTube Video Links

List of Figures

- Figure 1:** ARMIAC robotic arm
- Figure 2:** ARMIAC robotic arm holding an object
- Figure 3:** Arduino IDE used to program the ESP32
- Figure 4:** Visual Studio Code used to program the Flutter application
- Figure 5:** Creation of the ARMIAC logo in Canva
- Figure 6:** Samsung S21 used for the application
- Figure 7:** PC used for programming
- Figure 8:** Laptop used for programming
- Figure 9:** Elegoo Neptune 4 3D printer
- Figure 10:** ESP32 microcontroller
- Figure 11:** PCA9685 module (16-channel PWM driver)
- Figure 12:** MZ996R servo
- Figure 13:** JX low-profile servo
- Figure 14:** 6V 10A DC power supply
- Figure 15:** Voltage measurement with a multimeter before powering the system
- Figure 16:** Servo extension cables used for the thumb servo
- Figure 17:** Servo extension cables in use for the thumb servo
- Figure 18:** Elegoo Neptune 4 3D printer
- Figure 19:** Assembly materials used in the project
- Figure 20:** System overview diagram
- Figure 21:** First contact with this type of project
- Figure 22:** 3D printing of parts
- Figure 23:** Beginning of hand assembly
- Figure 24:** First test with a servomotor
- Figure 25:** Internal section of the arm
- Figure 26:** Creating the hand in TinkerCAD
- Figure 27:** Assembled arm
- Figure 28:** Application on the phone - earlier version
- Figure 29:** Programming the application in Dart
- Figure 30:** Creating the elbow in TinkerCAD
- Figure 31:** Hand base
- Figure 32:** Fingers
- Figure 33:** Opposable thumb
- Figure 34:** Wrist
- Figure 35:** Forearm
- Figure 36:** Elbow
- Figure 37:** Servo support
- Figure 38:** Servo attachments
- Figure 39:** Forearm cover
- Figure 40:** Forearm print failure
- Figure 41:** Finger assembly
- Figure 42:** Fingers mounted on the hand base

Figure 43: Insertion of the thumb servo
Figure 44: Mounting the opposable thumb on the hand
Figure 45: Nylon line used to pull the fingers
Figure 46: Wrist assembly
Figure 47: Completed hand assembly
Figure 48: Servos mounted on the support
Figure 49: Servo horn with 3D-printed attachments
Figure 50: Servo horn attachment fitted to the servos
Figure 51: Forearm servo horn
Figure 52: Mounting the servo support in the forearm
Figure 53: Distribution of servomotors across PCA9685 PWM channels
Figure 54: Servo extension cables used for the thumb servo
Figure 55: Nylon lines attached to the servo attachments
Figure 56: ARMIAC base
Figure 57: Voltage measurement with a multimeter before powering the system
Figure 58: Connection diagram between the ESP32 and PCA9685 module
Figure 59: Connecting the power supply to the PCA9685
Figure 60: Application home screen
Figure 61: Application mini-tutorial slides - 1
Figure 62: Application mini-tutorial slides - 2
Figure 63: Application control panel
Figure 64: List of available voice commands
Figure 65: Settings page
Figure 66: Terms of service page
Figure 67: Creating the Flutter project
Figure 68: Flutter project folder structure
Figure 69: ARMIAC logo
Figure 70: Completed project

Acknowledgements

First of all, I would like to thank my supervising teachers, Cláudia Barata and Paula Alcobia, for their guidance, support, and suggestions throughout the development of the project.

I would also like to thank all the teachers who contributed to my education during the Professional Course in Computer Systems Management and Programming, providing important knowledge for the completion of this work, particularly teacher Luís Roque for providing access to the 3D printing room available at the school.

I would also like to thank my family, especially my mother, for her support and encouragement throughout the entire project development process.

Finally, I would like to thank all my classmates and everyone who, in some way, contributed to the development of ARMIAC through opinions, ideas, and feedback.

1 - Introduction

The PAP, Professional Aptitude Project, is a final course project whose purpose is to demonstrate the skills acquired during the three years of training in the Professional Course in Computer Systems Management and Programming.

The project chosen for the PAP presentation is entitled **"ARMIAC - Intelligent Arm."** I chose this topic because of my interest in robotics, programming, electronics, and the simulation of human movement.

ARMIAC consists of a 3D-printed robotic hand controlled by servomotors and by a mobile application developed in Flutter. The application can send commands to open and close the hand, control individual movements, and perform different gestures through a Wi-Fi connection, using either voice commands or buttons.

The system uses an ESP32 as its main controller, responsible for receiving the commands sent by the application, and a PCA9685 module to control the servomotors. In this way, the commands sent by the user through the application are transformed into physical movements of the robotic hand.

The mobile application was developed for Android and includes a simple and organized interface with control buttons, quick commands, a movement panel, voice commands, a usage tutorial, Portuguese and English language support, a list of available commands, and a terms of service page. The latest version of the project is **1.1.7**.

The project was developed in several stages, from the initial research to the final prototype tests. The project was first planned and similar solutions were studied. The 3D parts were then created and printed, the physical structure was assembled, the servomotors were installed, and the electronic connections were made. Finally, the ESP32 was programmed, the mobile application was developed, and functional tests were performed.

This report presents the technologies used, the components applied, the prototype development process, the system implementation, the difficulties encountered, the results obtained, and possible future improvements.

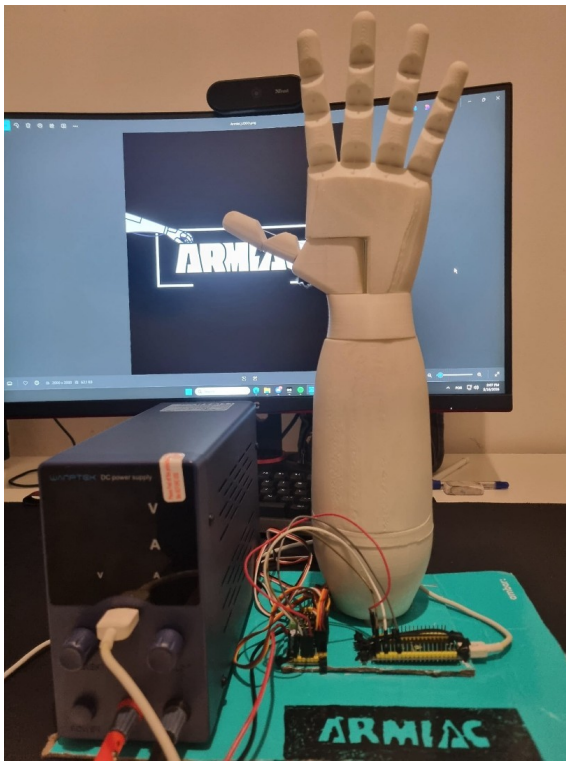


Figure 1: ARMIAC robotic arm

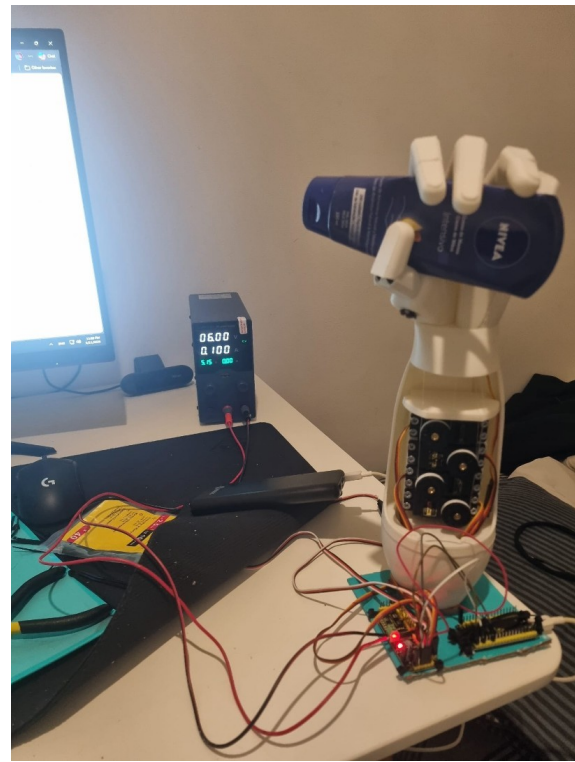


Figure 2: ARMIAC robotic arm holding an object

1.1 - Objective and Real-World Application in Education

The main objective of ARMIAC is to develop a functional robotic hand prototype controlled by a mobile application, demonstrating the connection between programming, electronics, robotics, 3D printing, and application development.

In a real educational setting, ARMIAC could be used as an educational tool in schools, professional courses, robotics clubs, or technology training programs. Through this project, students could learn in a practical way how communication between software and hardware works, understanding how a mobile application can send commands to a microcontroller and control physical movements through servomotors.

The project also makes it possible to explore content from several areas, including Dart and C++ programming, Flutter application development, Wi-Fi networks, HTTP communication, PWM control, 3D modelling, 3D printing, and electronic assembly. ARMIAC can therefore serve as a practical example in programming, electronics, and robotics classes, making learning more visual, interactive, and closely connected to real situations.

In addition, because it is a physical prototype, ARMIAC helps users understand the errors and challenges that arise during the development of a technological system, such as power supply problems, electrical connections, servo calibration, code organization, and final testing. The project therefore demonstrates not only a final product, but also the complete process of creating, testing, and improving a technological solution.

2 - Technologies Involved

Several technologies and tools, at both software and hardware levels, were required to develop this project.

2.1 - Software Used

Software used to write the report:

- LibreOffice

Programming software (IDE):

- Arduino IDE
- Visual Studio Code / Android Studio

Software used to create the 3D models:

- TinkerCAD

Software used for 3D printing:

- Elegoo Cura / Ultimaker Cura

Software used for graphic design:

- Canva (ARMIAC logo and system overview diagram)

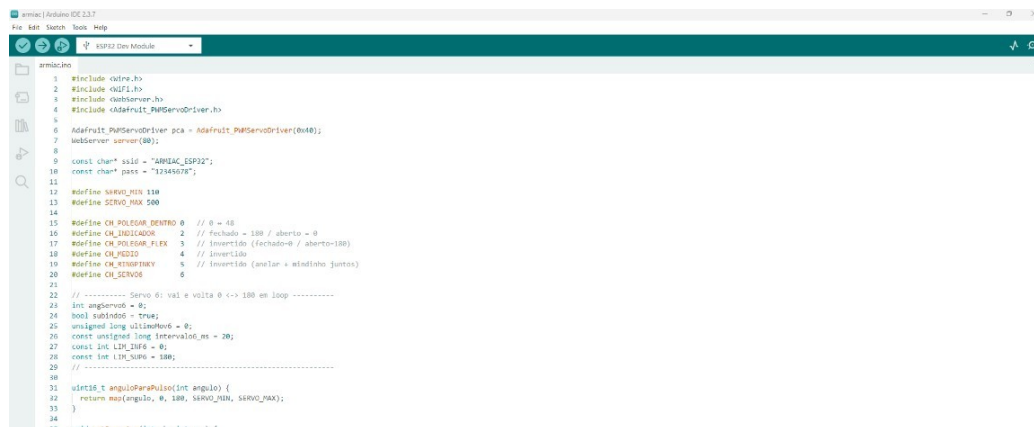


Figure 3: Arduino IDE used to program the ESP32

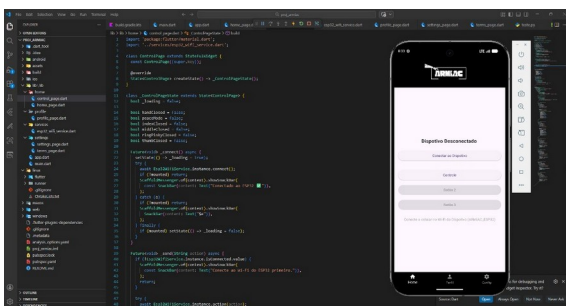


Figure 4: Visual Studio Code used to program the Flutter application

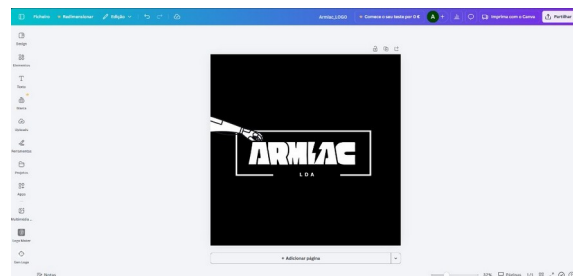


Figure 5: Creation of the ARMIAC logo in Canva

2.2 - Devices Used

- **Desktop computer and laptop** - used to program the ESP32, develop the Flutter application, write the report, and prepare the 3D files.
- **Samsung S21 smartphone** - used to test the ARMIAC application and send commands to the robotic arm.
- **Elegoo Neptune 4 3D printer** - used to print the hand, forearm, and base parts.

Figura 6: Samsung S21 utilizado para a APP



Figure 6: Samsung S21 used for the application

Figura 7: PC utilizado para programação



Figure 7: PC used for programming



Figure 8: Laptop used for programming



Figure 9: Elegoo Neptune 4 3D printer

2.3 - Programming Languages

- **C++ (Arduino / ESP32)** - used to program the ESP32 through the Arduino IDE. This language was used to create the ARMIAC Wi-Fi network, receive HTTP commands, control the PCA9685, and execute the servomotor movements.
- **Dart (Flutter)** - used to develop the mobile application with Flutter. This language made it possible to create the graphical interface, command system, voice control, Portuguese and English translation support, and communication with the ESP32.

2.4 - Technologies and Communication

ARMAC uses different communication technologies to connect the mobile application to the physical system.

- **Wi-Fi** - the ESP32 creates its own Wi-Fi network in Access Point mode, named "**ARMAC**". The smartphone connects directly to this network without requiring a router or internet connection.
- **HTTP protocol** - the mobile application sends commands to the ESP32 through HTTP. These commands are sent through URLs such as `/action?a=open_hand``, which tells the ESP32 to open the hand.
- **I2C/PWM** - I2C communication is used between the ESP32 and PCA9685 through the SDA and SCL pins. The PCA9685 receives instructions from the ESP32 and generates the PWM signals required to control the servomotors.

2.5 - Electronic Components Used

ESP32

The ESP32 is the microcontroller used as the main controller of the system. It is responsible for creating the ARMIAC Wi-Fi network, receiving commands sent by the mobile application, and communicating with the PCA9685 controller.

It works as the "brain" of the system, but it does not directly control all of the servomotors. Instead, it sends instructions to the PCA9685, which is responsible for generating the PWM signals for the servos.



Figure 10: ESP32 microcontroller

PCA9685

The PCA9685 is a 16-channel PWM controller used to control several servomotors in a more stable way. Since the project uses several servos to move the fingers and forearm, the PCA9685 makes the connections easier to organize.



Figure 11: PCA9685 module (16-channel PWM driver)

Servomotors

The servomotors are responsible for the physical movement of the fingers and forearm. Each servo receives a PWM signal and moves to a specific angle, allowing the fingers to open or close.

MZ996R Servo

This servo is used to flex the fingers and rotate the forearm.



Figure 12: MZ996R servo

JX Low-Profile Servo

This servo is used specifically to move the thumb inward.



Figure 13: JX low-profile servo

6V 10A DC Power Supply

This power supply is used to power all the servomotors and provide stable current.



Figure 14: 6V 10A DC power supply

Multimeter

During the development of the project, a digital multimeter was used to measure voltage and ensure safety before connecting the components to the power supply.



Figure 15: Voltage measurement with a multimeter before powering the system

Servo cables and extensions were also required during the assembly and connection of the servomotors to ensure correct connections between the servos and the PCA9685.

Servo extension cables were used mainly for the low-profile thumb servo because the position of this servo required a longer cable.

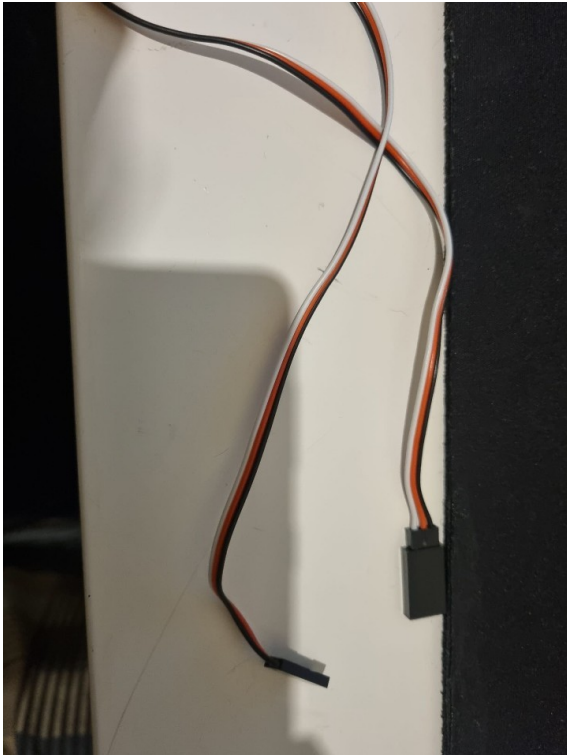


Figure 16: Servo extension cables used for the thumb servo

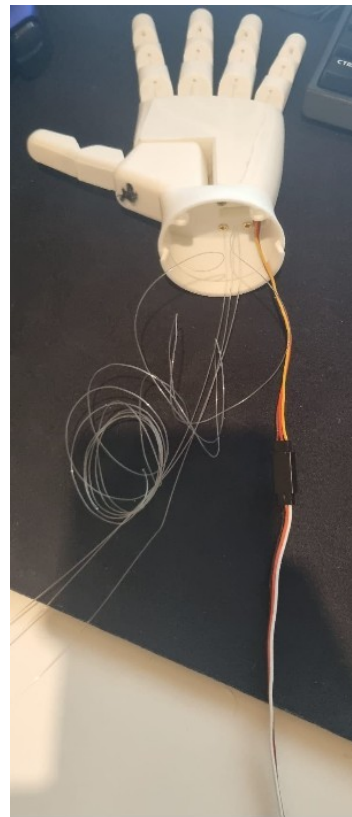


Figure 17: Servo extension cables in use for the thumb servo

2.6 - 3D Printing and Assembly Materials

The prototype was built using parts printed in PLA on an Elegoo Neptune 4 3D printer.



Figure 18: Elegoo Neptune 4 3D printer

Materials used:

- 3 mm black elastic cord
- 0.4 mm nylon line
- Screws
- Superglue



Figure 19: Assembly materials used in the project

Component / Material	Approx. Quantity	Function in the Project
ESP32	1	Main system controller. Creates the Wi-Fi network, receives application commands, and communicates with the PCA9685.
PCA9685	1	PWM controller used to control several servomotors in an organized and stable way.
MZ996R servomotors	6	Responsible for moving the fingers and

		forearm.
JX low-profile servo	1	Used to move the thumb inward.
6V 10A DC power supply	1	Provides sufficient and stable power to the servomotors.
Servo extension cables	1	Allow servos located farther away to be connected to the PCA9685.
0.4 mm nylon line	As required	Used to pull the fingers, simulating tendons.
3 mm black elastic cord	As required	Used in the finger joints to assist movement.
Screws	32	Used to fasten printed parts and components.
Superglue	1	Used to secure some structural parts.
White PLA	As required	Material used for 3D printing the parts.
Elegoo Neptune 4 3D printer	1	Used to print the hand, forearm, and base parts.
Samsung S21 smartphone	1	Used to test and operate the ARMIAC application.
Computer / laptop	1 or more	Used for ESP32 programming, application development, and preparation of 3D files.
Digital multimeter	1	Used to measure supply voltage and verify electrical connections.

3 - Project Implementation

Theoretical Framework

ARMIAC can be divided into four main parts: the **mobile application**, **ESP32**, **PCA9685**, and **servomotors**.

Robotic Arm

A robotic arm is a mechanical and electronic system capable of simulating human movements. These systems are frequently used in industrial, medical, and technological fields.

In the ARMIAC project, a robotic arm prototype with an articulated hand was developed, allowing several movements of the hand and forearm.

Servomotors

Servomotors allow an angle to be controlled accurately, generally between 0° and 180°. ARMIAC uses servos to control the movement of each finger.

PWM

PWM (Pulse Width Modulation) is a technique used to control the position of servos through electrical signals with different pulse widths.

Wi-Fi and HTTP Communication

Remote control is performed through a Wi-Fi network created by the ESP32. The Flutter application sends HTTP commands that are interpreted by the microcontroller.

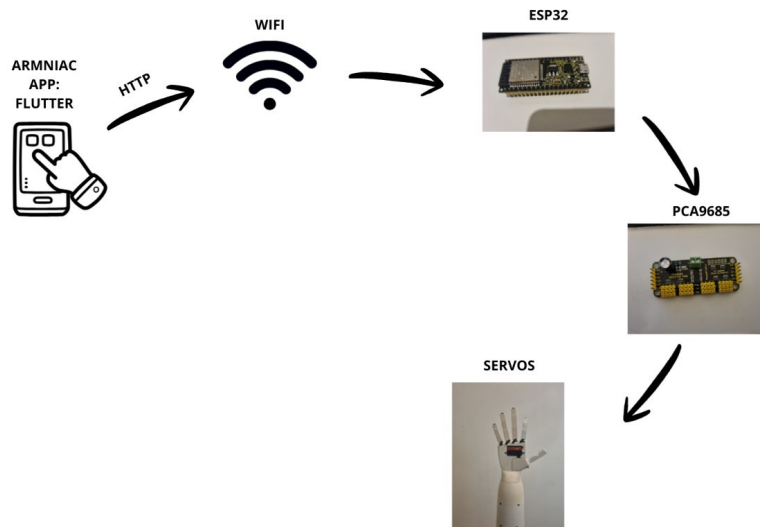


Figure 20: System overview diagram

4 - Project Development

4.1 - Timeline and Development Phases

The development of **ARMNIAC - Intelligent Arm** took place throughout the 2025/2026 academic year, following an organized sequence of stages from the initial research phase through the development of the mobile application and the final tests of the prototype.

The project was planned progressively. The idea and possible technical solutions were structured first; the parts were then created, followed by 3D printing, physical and electronic assembly, and finally software implementation.

The main phases of the project were as follows.

Phase 1 - Research and Planning (5 October)

On 5 October, the research and planning phase began. Ideas were analysed and similar projects were studied in order to understand which components would be necessary and how a functional robotic hand could be developed.

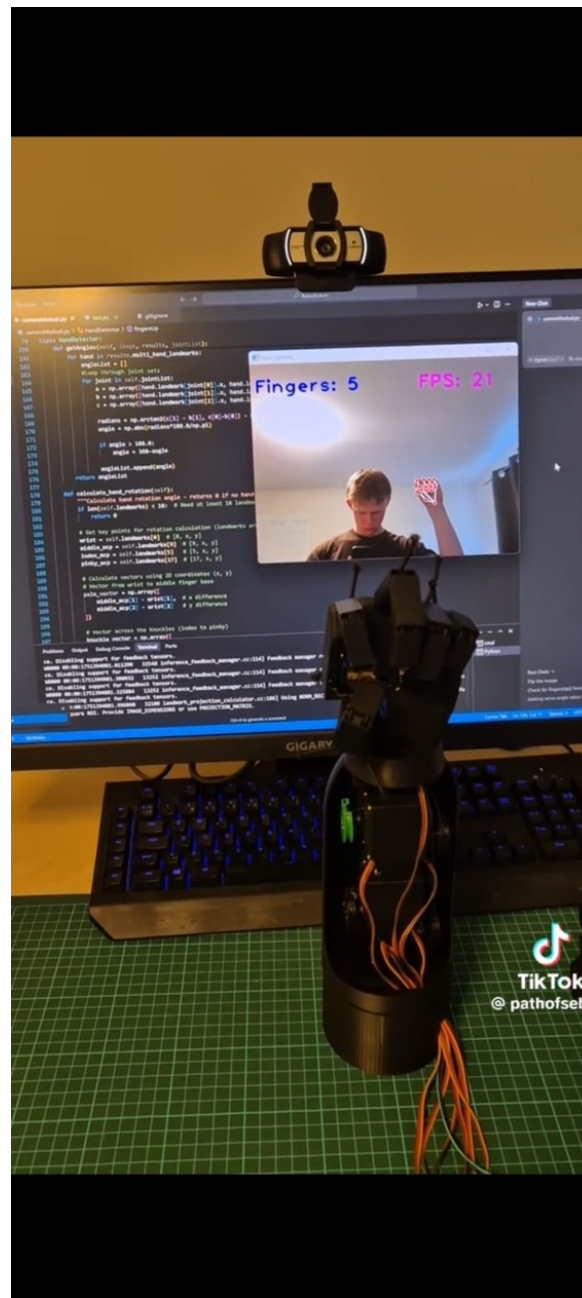


Figure 21: First contact with this type of project

A demonstration of two base projects is included under "**Example of base projects**" in the appendices, section 7.3.

Phase 2 - 3D Printing of the Parts - First Version (20 October)

On 20 October, 3D printing of the project parts began. During this phase, the first structures required to assemble the hand and forearm were produced using two Elegoo Neptune 4 printers.

A first version of 3D parts found online was used as an initial basis for better understanding the structure of a robotic hand and the operation of its joints. This version provided a first contact with the shape of the parts, the fittings, and the general assembly of the prototype.

However, after printing and analysing the result, it became clear that the initial version did not fully match what was intended for ARMIAC. For this reason, a custom version of the parts was created, using the first version only as a reference.



Figure 22: 3D printing of parts

A demonstration is included under **"3D Printing"** in the appendices, section 7.3.

Phase 3 - Initial Hand Assembly (29 October)

On 29 October, assembly of the robotic hand began, including the finger joints and the initial adjustment of the printed parts.



Figure 23: Beginning of hand assembly

Phase 4 - First Servomotor Test (5 November)

On 5 November, the first test with a mini servomotor was performed in order to validate the mechanical operation of the system and test the ability to move one finger.

This test was essential to confirm that the line-and-joint mechanism worked correctly.

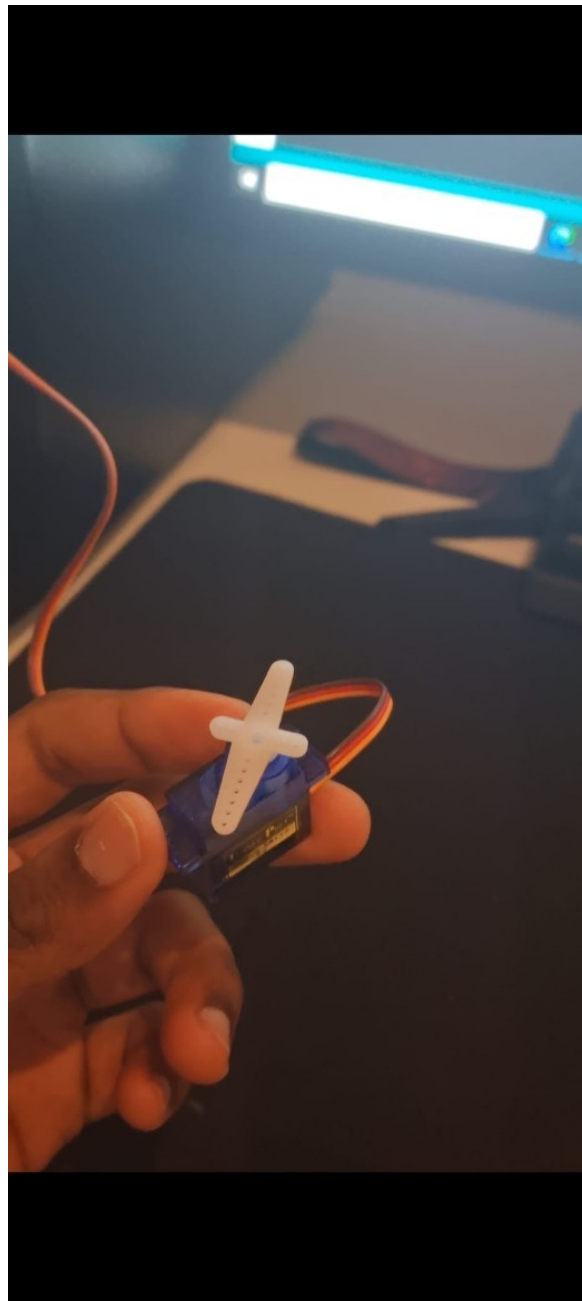


Figure 24: First test with a servomotor

A demonstration is included under "**First servomotor test**" in the appendices, section 7.3.

Phase 5 - Testing and Forearm Assembly (14 to 16 November)

Between 14 and 16 November, several additional tests were carried out and assembly of the remaining forearm began, including the installation of additional servos and organization of the internal space.

During this phase, adjustments were made to the line system and electrical connections, providing greater stability and functionality to the prototype.

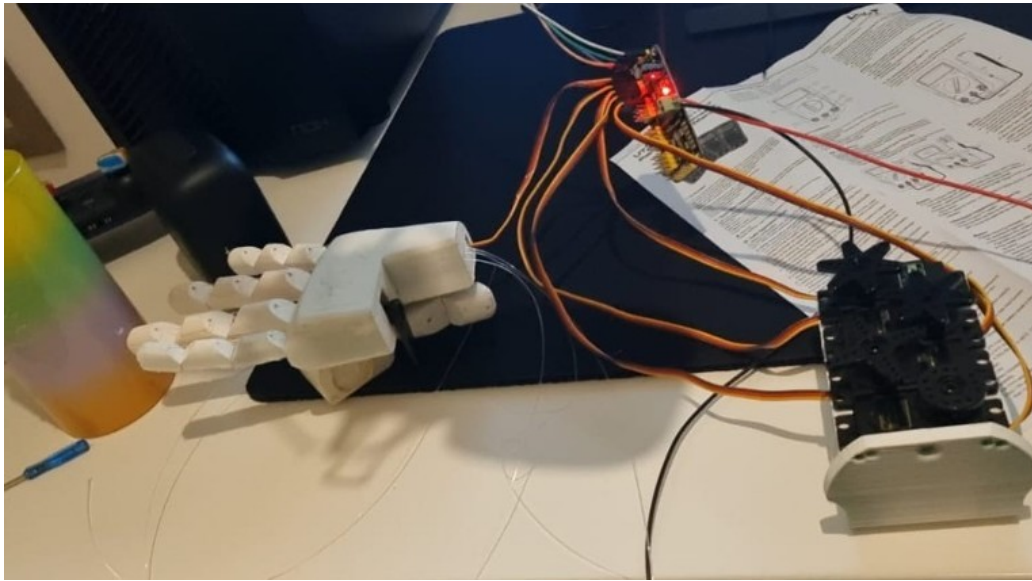


Figure 25: Internal section of the arm

Phase 6 - Creation and Printing of a Custom Version of the 3D Parts (29 November to the End of December)

From 29 November through the Christmas period, work began on creating a custom version of the ARMIAC 3D parts, using the first version found online as a reference. After the initial tests, it became clear that the parts needed to be adapted to the project by improving the fittings, fingers, and routing of the lines.

It was also decided to print the parts in white PLA because the visual result was cleaner and more suitable for the final prototype.

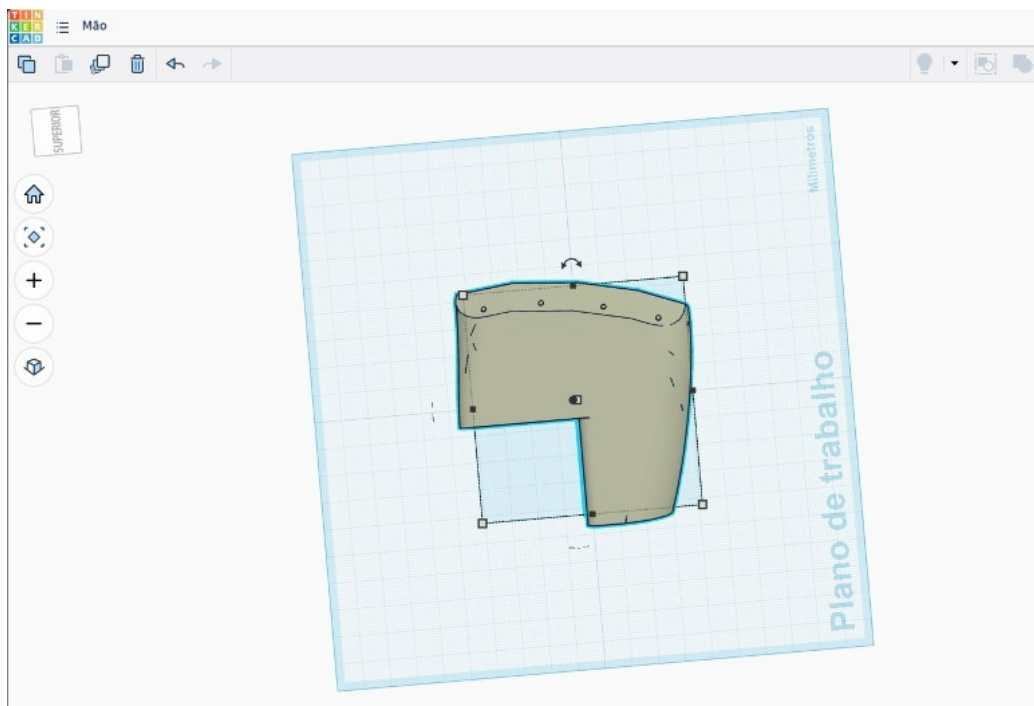


Figure 26: Creating the hand in Tinkercad

Phase 7 - Final Forearm Assembly (20 January)

By 20 January, the forearm had been fully assembled, including installation of the servomotors, attachment of the parts, and preparation for final testing and calibration.



Figure 27: Assembled arm

Phase 8 - Development of the Mobile Application (21 January)

On 21 January, development of the mobile application in Flutter began, with the aim of creating a simple and functional interface for remote control of the robotic arm through Wi-Fi.

During this phase, control pages, action buttons, and an HTTP communication system between the smartphone and the ESP32 were created.



Figure 28: Application on the phone - earlier version

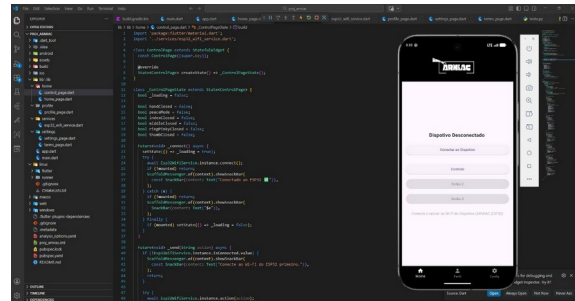


Figure 29: Programming the application in Dart

On 17 May, a demonstration of the ARMIAC prototype was carried out together with the mobile application developed in Flutter. Final tests of the complete system were performed, checking the Wi-Fi communication between the smartphone and the ESP32 and the response of the servomotors to the commands sent by the application.

After validating the operation of the prototype, a demonstration video was recorded showing control of the robotic arm through the application and execution of the configured movements and gestures.

A demonstration is included under "**Demonstration of the unfinished project**" in the appendices, section 7.3.

4.2 - Prototype Development

4.2.1 - Creation of the 3D Models

Before printing, some of the ARMIAC parts were modelled in 3D. TinkerCAD, an online 3D modelling tool, was used during this phase.

The parts were created based on observation of similar projects found online, which served as references for understanding the structure of a robotic hand. However, the parts were adapted to the needs of ARMIAC, taking into account the size of the servomotors, line routing, fittings, and final assembly.

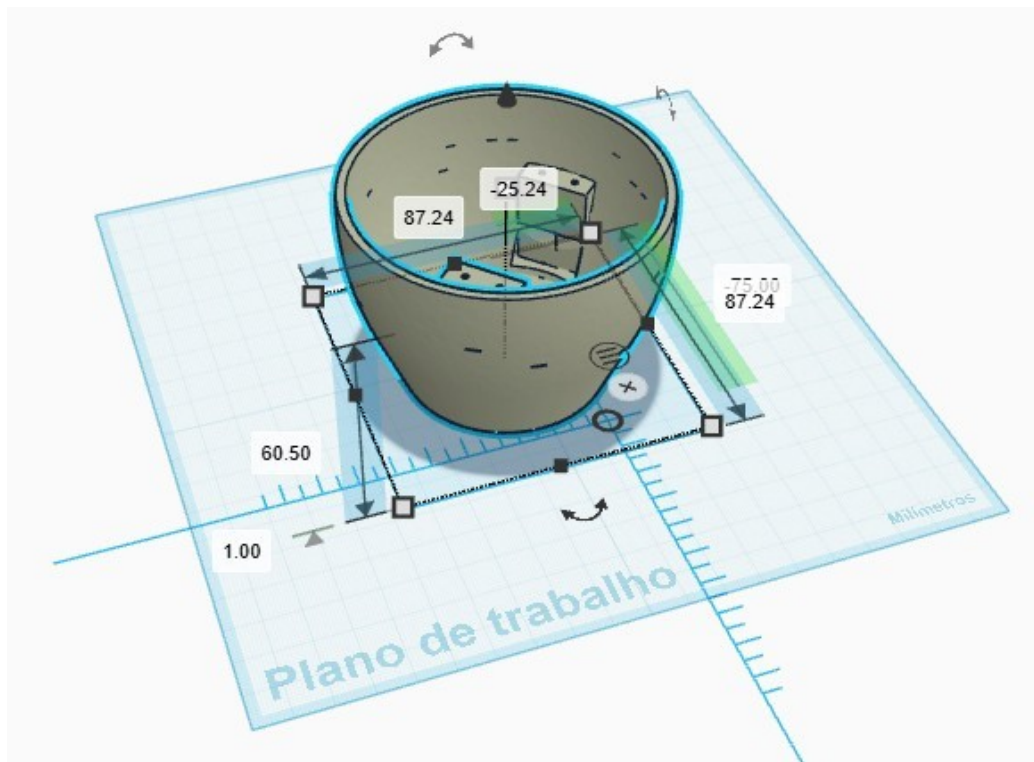


Figure 30: Creating the elbow in TinkerCAD

4.2.2 - Preparation of the 3D Models

During this stage, the 3D models required for the project were adjusted and prepared for printing, including the hand and forearm structure in STL format. The files were subsequently exported in G-code format.

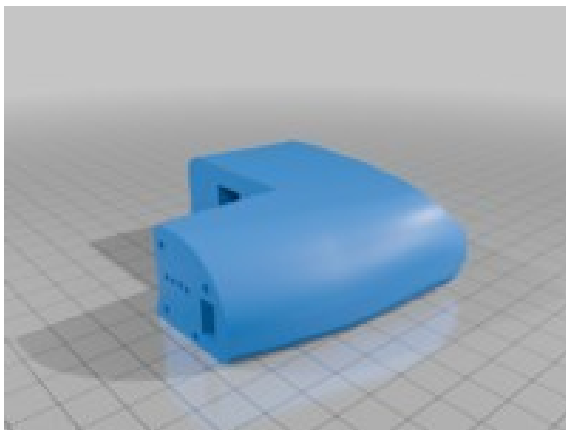


Figure 31: Hand base

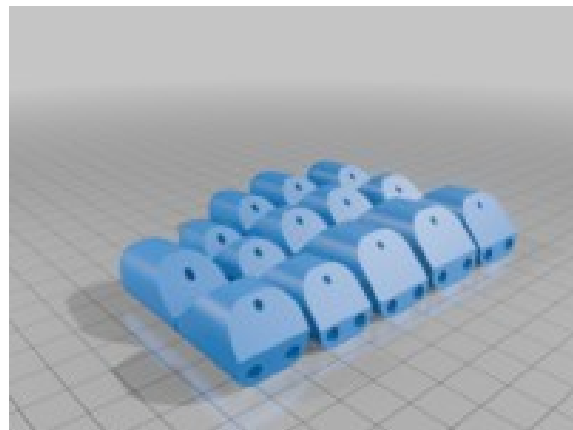


Figure 32: Fingers

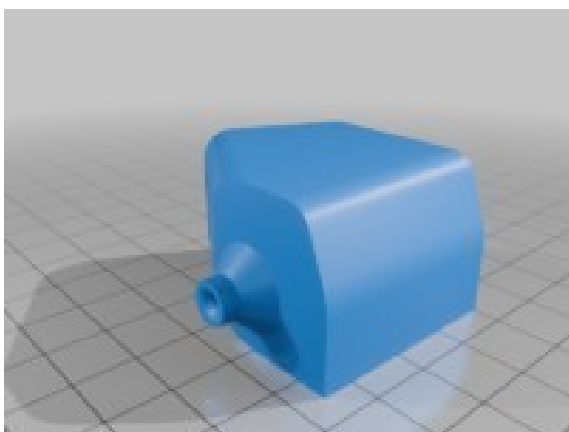


Figure 33: Opposable thumb

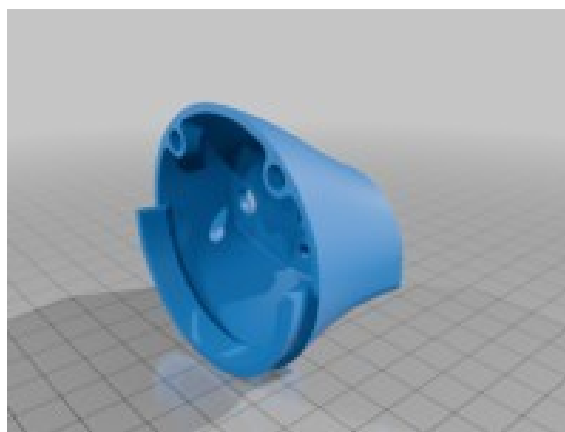


Figure 34: Wrist

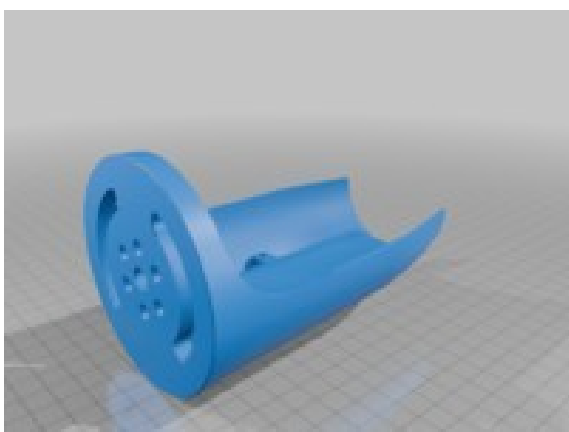


Figure 35: Forearm

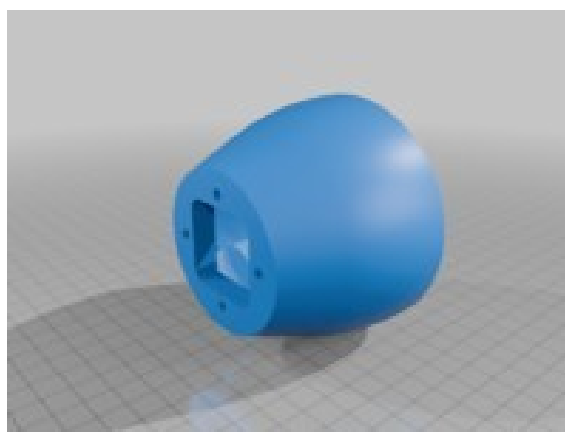


Figure 36: Elbow

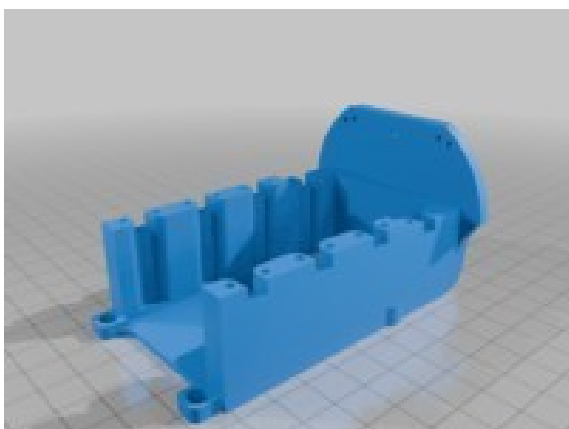


Figure 37: Servo support

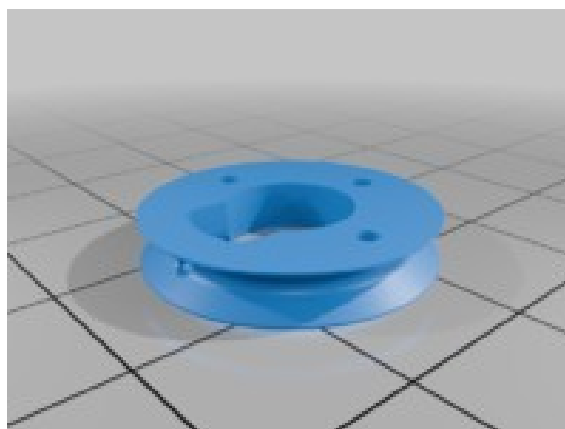


Figure 38: Servo attachments

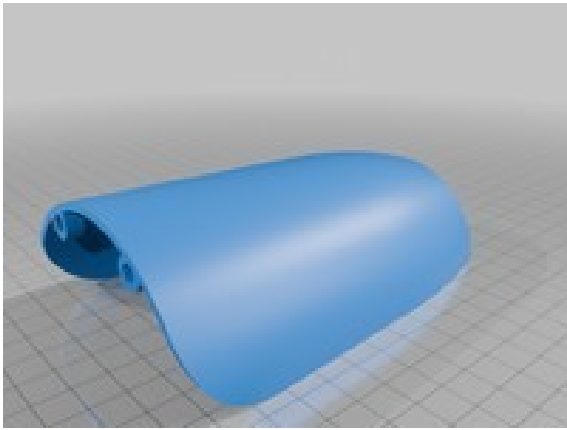


Figure 39: Forearm cover

4.2.3 - 3D Printing of the Parts

After the 3D models had been created and prepared, the parts were printed using the Elegoo Neptune 4 printer.



Figure 40: Forearm print failure

4.2.4 - Assembly of the Fingers and Joints

The fingers were assembled using 3 mm elastic cord, which allows flexion and simulates the natural movement of a human hand.

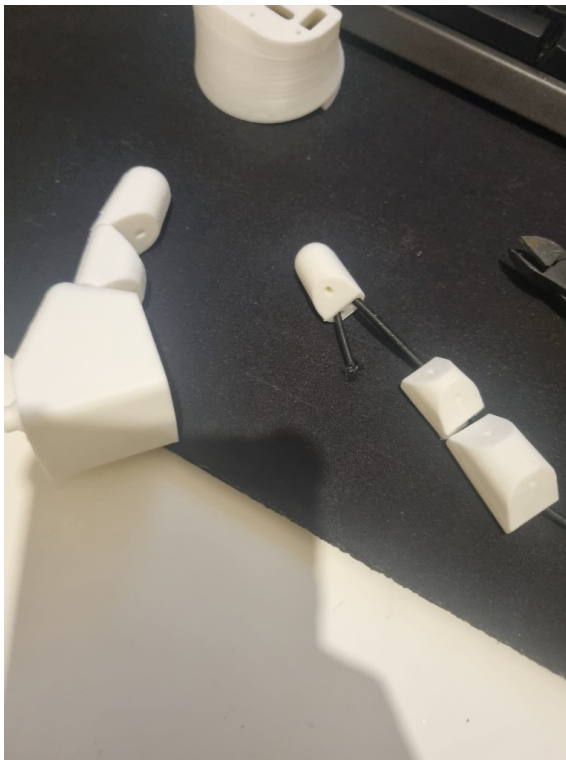


Figure 41: Finger assembly



Figure 42: Fingers mounted on the hand base

A demonstration is included under "**Demonstration of finger operation**" in the appendices, section 7.3.

4.2.5 - Assembly of the Hand and Wrist

Nylon lines were passed through the inside of the fingers and the thumb servo was fitted into position.



Figure 43: Insertion of the thumb servo

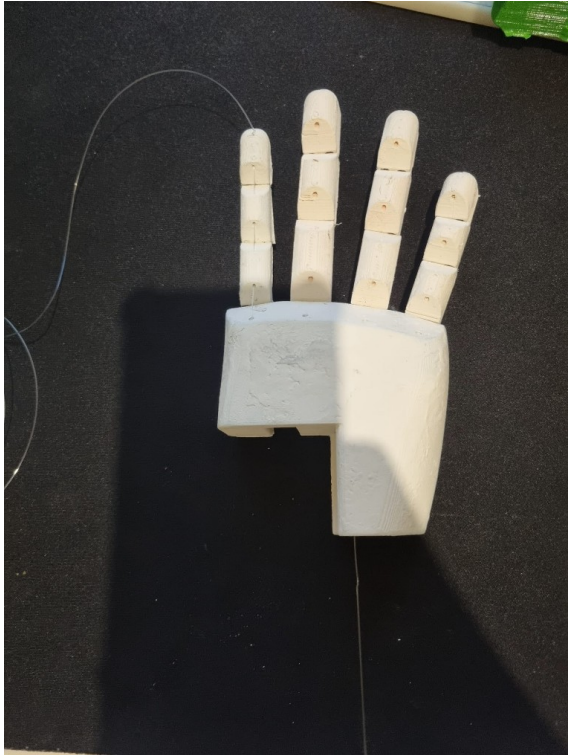


Figure 44: Mounting the opposable thumb on the hand

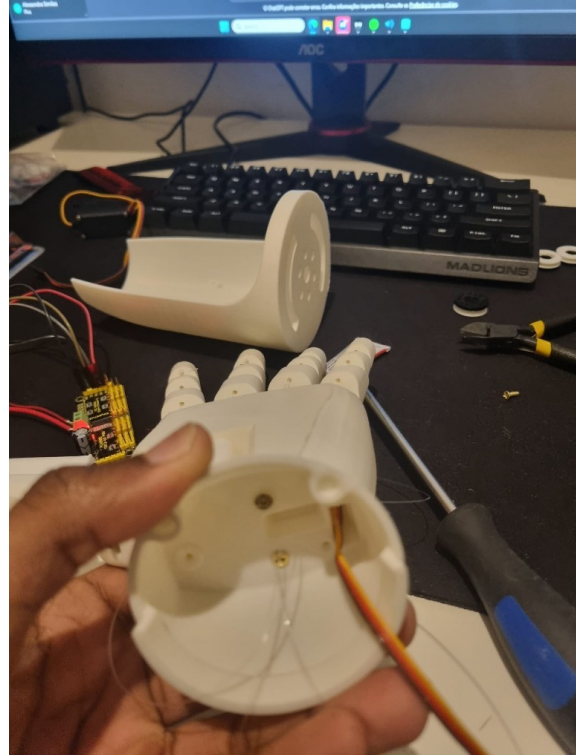


Figure 45: Nylon line used to pull the fingers

Figure 46: Wrist assembly

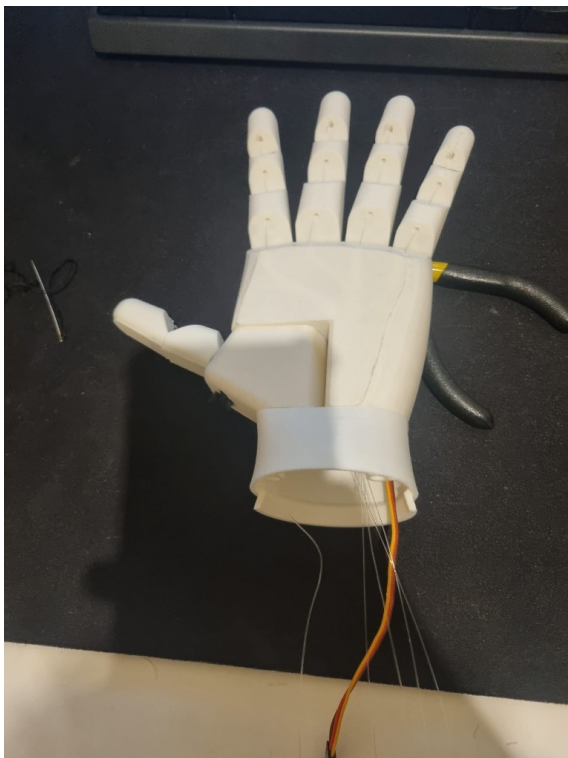


Figure 47: Completed hand assembly

4.2.6 - Installation of the Forearm Servomotors

Several MZ996R servomotors were installed to control the fingers and forearm rotation, together with a low-profile servo for the thumb.



Figure 48: Servos mounted on the support



Figure 49: Servo horn with 3D-printed attachments



Figure 50: Servo horn attachment fitted to the servos



Figure 51: Forearm servo horn



Figure 52: Mounting the servo support in the forearm

The servomotors were connected to the PCA9685 PWM channels in an organized way, allowing each movement of the robotic hand to be controlled individually. PWM channel 0 was assigned to the base of the opposable thumb, PWM 1 to the opposable thumb, PWM 2 to the index finger, PWM 3 to the middle finger, PWM 4 to the ring and little fingers, and PWM 5 to forearm movement.

This organization simplified the programming of movements on the ESP32 and made it easier to identify each servo during testing.

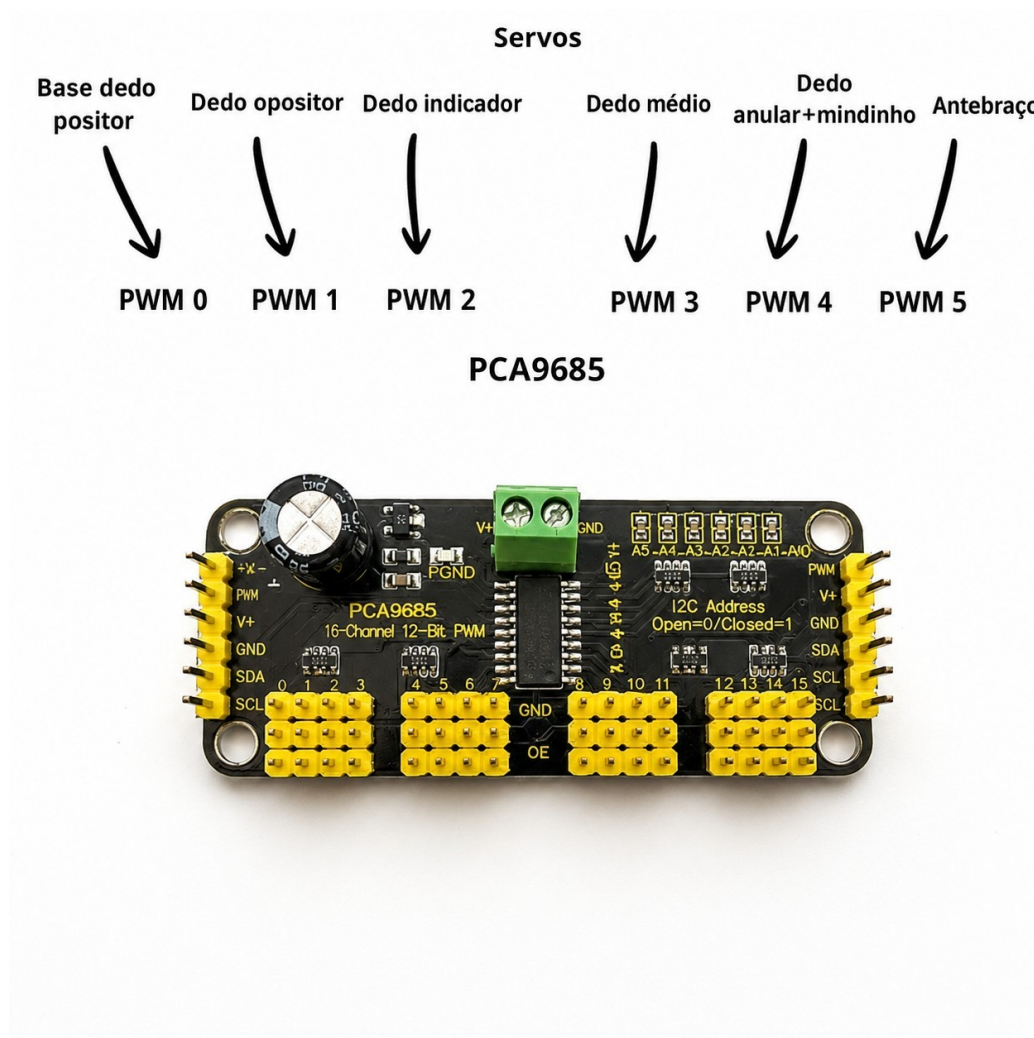


Figure 53: Distribution of servomotors across PCA9685 PWM channels

PCA9685 Channel	Controlled Movement
PWM 0	Base of the opposable thumb
PWM 1	Opposable thumb
PWM 2	Index finger
PWM 3	Middle finger
PWM 4	Ring + little fingers
PWM 5	Forearm

4.2.7 - Cables and Extensions

Because of the distance between some servos and the PCA9685 controller, servo cable extensions were used, especially for the low-profile thumb servo.

This stage was important for ensuring stable connections and better internal organization of the arm.

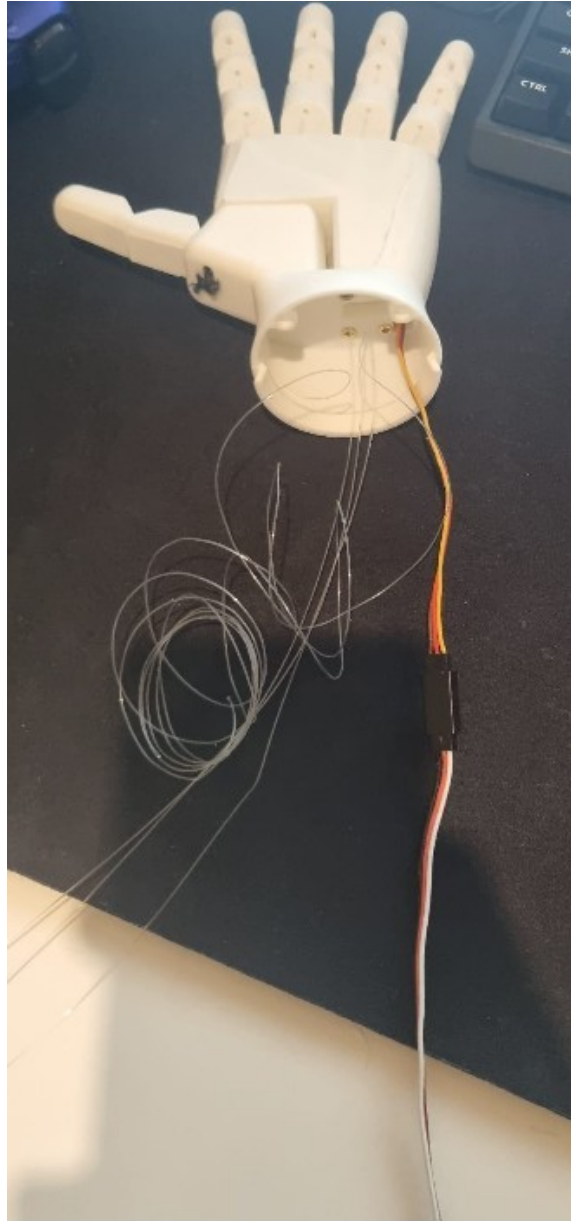


Figure 54: Servo extension cables used for the thumb servo

4.2.8 - Fitting the Hand and Positioning the Lines

At this stage, the hand was fitted to the forearm and the lines were passed through the spaces in the servo support, where they were secured and wound around the servos.



Figure 55: Nylon lines attached to the servo attachments

4.2.9 - Creation of the ARMIAC Base

A base was created to improve the presentation and portability of the project.

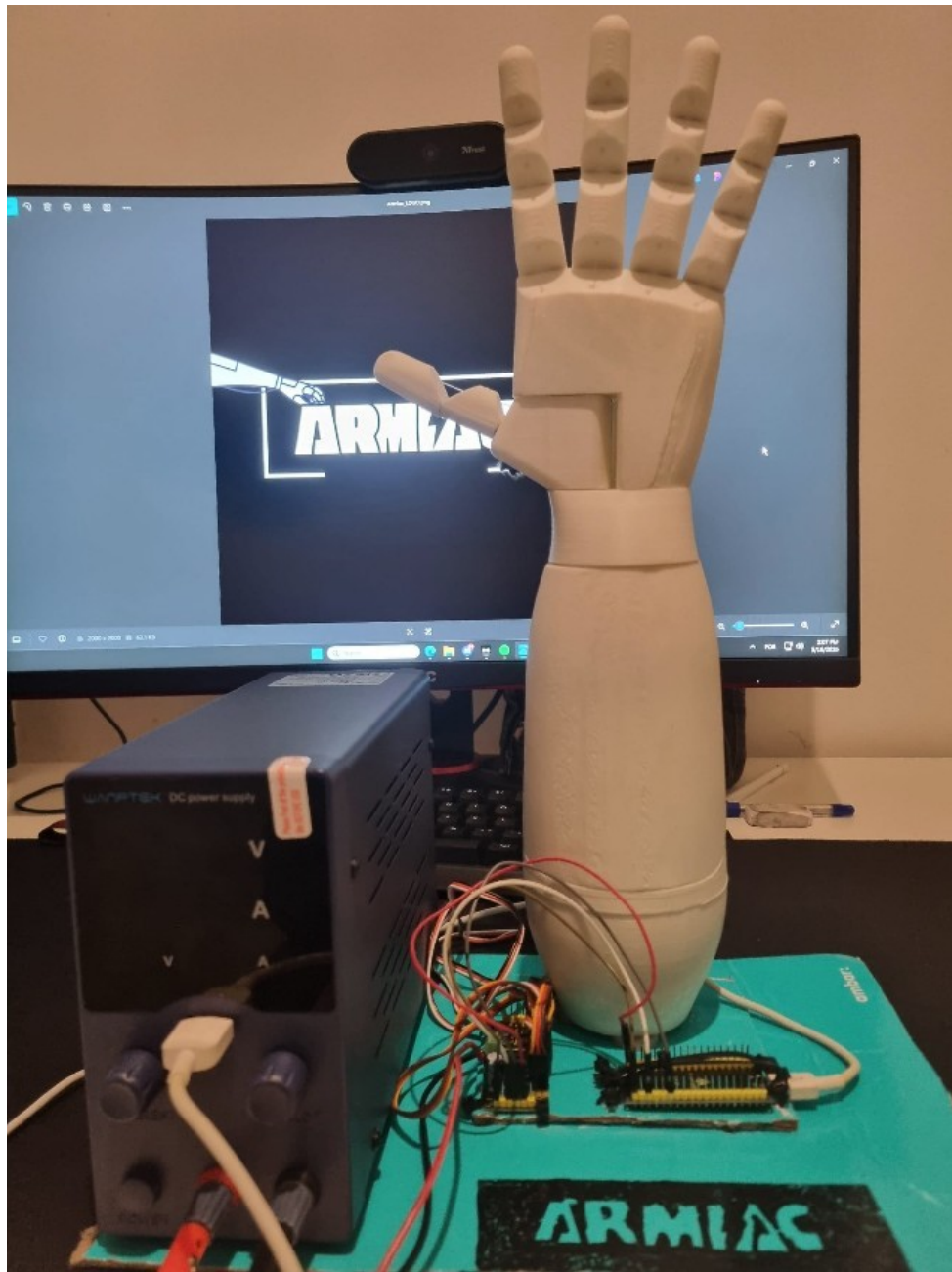


Figure 56: ARMIAC base

4.2.10 - Electrical Tests and Multimeter Measurements

Before connecting the complete system to the power supply, a multimeter was used to measure the voltage and confirm that the value was correct (**6 V**). This check increased system safety and reduced the risk of damaging components.



Figure 57: Voltage measurement with a multimeter before powering the system

4.2.11 - Connections Between the ESP32 and PCA9685

At this stage, the ESP32 was connected to the PCA9685 module using I2C communication. The ESP32 **3V3** pin was connected to the **VCC** pin of the PCA9685 to power the module's logic circuitry. The ESP32 **GND** pin was connected to the PCA9685 **GND**, providing a common reference between the components.

For communication, **GPIO21** was used as **SDA** and **GPIO22** as **SCL**. In this way, the ESP32 can send instructions to the PCA9685, which then generates the PWM signals required to control the servomotors.

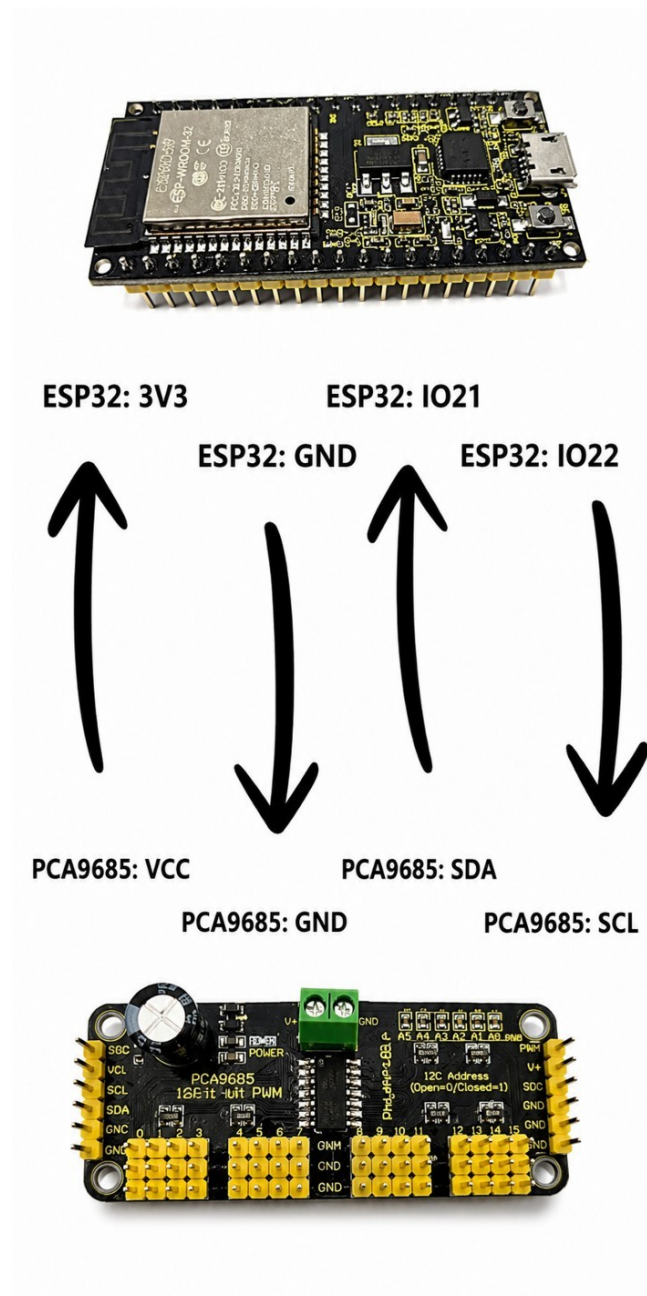


Figure 58: Connection diagram between the ESP32 and PCA9685 module

4.2.12 - Power Supply and Initial Tests

After the mechanical and electronic assembly, the system was powered using the **6V 10A DC power supply**. Initial tests were carried out to verify whether all the servos responded correctly to the commands.

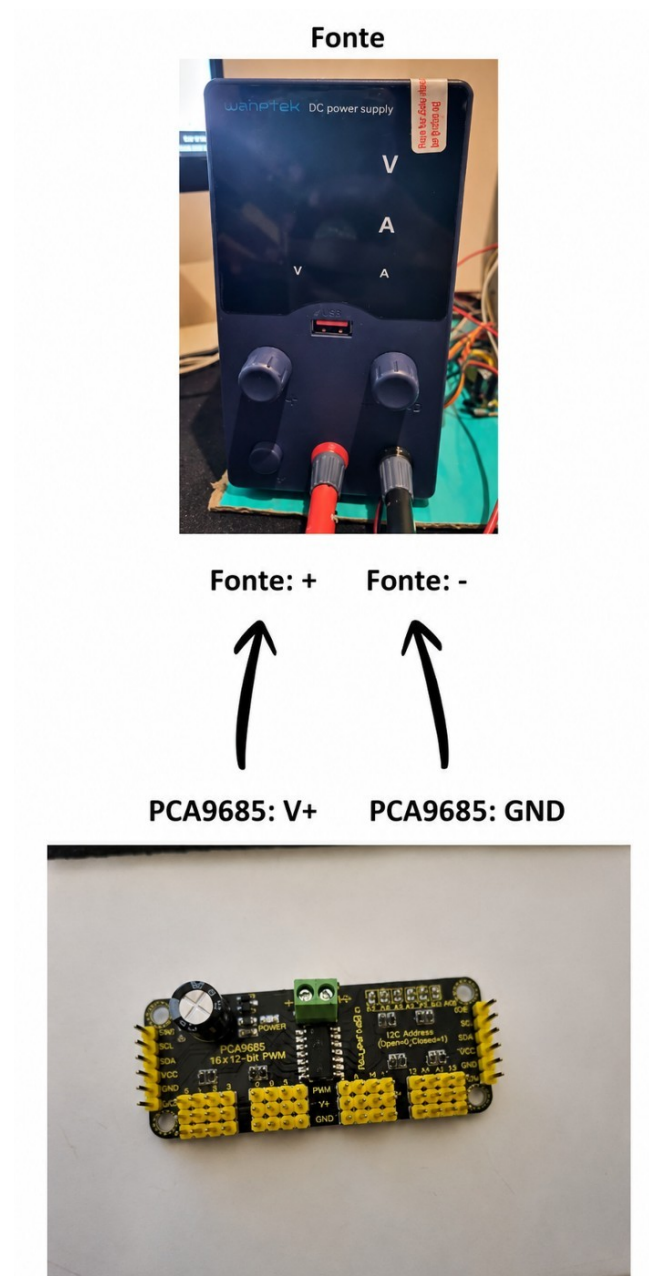


Figure 59: Connecting the power supply to the PCA9685

A demonstration is included under "**Demonstration of finger operation**" in the appendices, section 7.3.

4.3 - ESP32 Programming (Arduino IDE)

The ESP32 was programmed using the Arduino IDE. The microcontroller was configured to create a Wi-Fi network in Access Point mode and receive commands through a web server.

HTTP routes were created to enable remote control of the robotic arm through the Flutter application.

4.4 - Flutter Application Programming

A mobile application was developed in Flutter to control ARMIAC, allowing HTTP commands to be sent to the ESP32.

The application includes a home page showing the connection status and a control page where the hand can be opened or closed and the fingers can be controlled individually.



Figure 60: Application home screen



Figure 61: Application mini-tutorial slides - 1

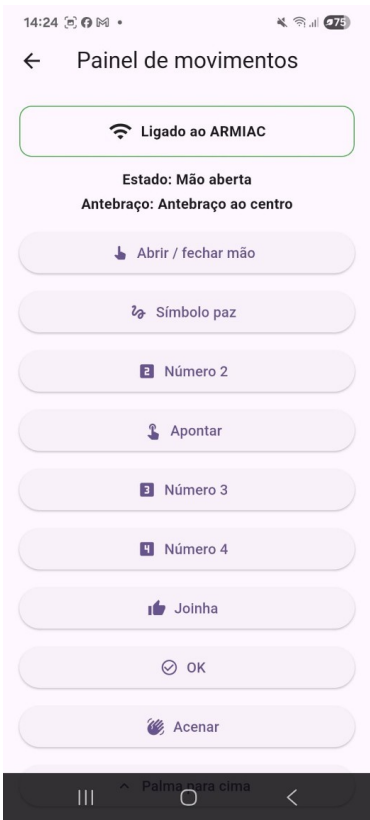


Figure 62: Application mini-tutorial slides - 2

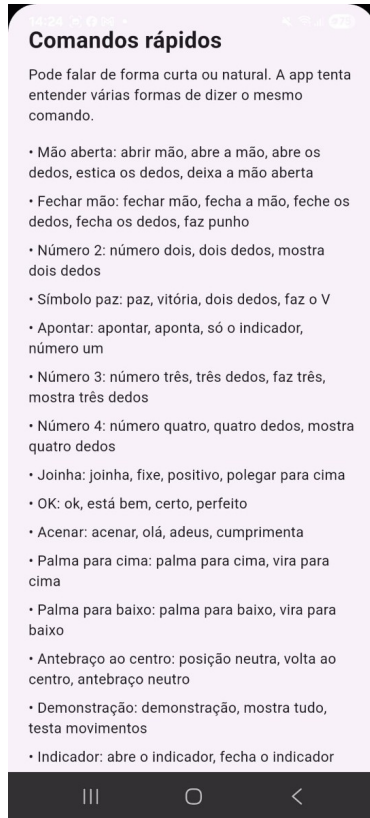


Figure 64: List of available voice commands

Figure 63: Application control panel



Figure 65: Settings page

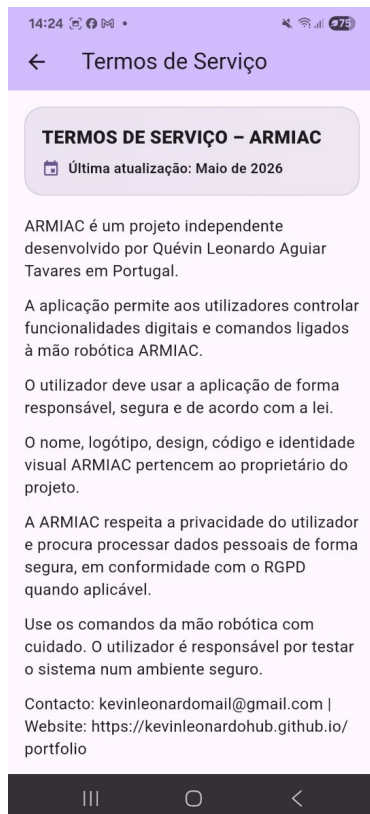


Figure 66: Terms of service page

4.4.1 - Creating the Flutter Project

To begin developing the ARMIAC mobile application, a Flutter project was created through the terminal. The Flutter SDK first had to be installed, the environment variables configured, and the installation checked with the command:

```
flutter doctor
```

Once the environment was configured correctly, the project was created with:

```
flutter create armiac_app
```

The project was then opened in Visual Studio Code, where the pages, services, and files required for the application were created. During development, commands such as `flutter pub get` were used to install dependencies and `flutter run` was used to test the application on an Android device.

The initial Flutter structure was then adapted for the ARMIAC project, separating the code into pages, services, and resources to make the application more organized and easier to maintain.

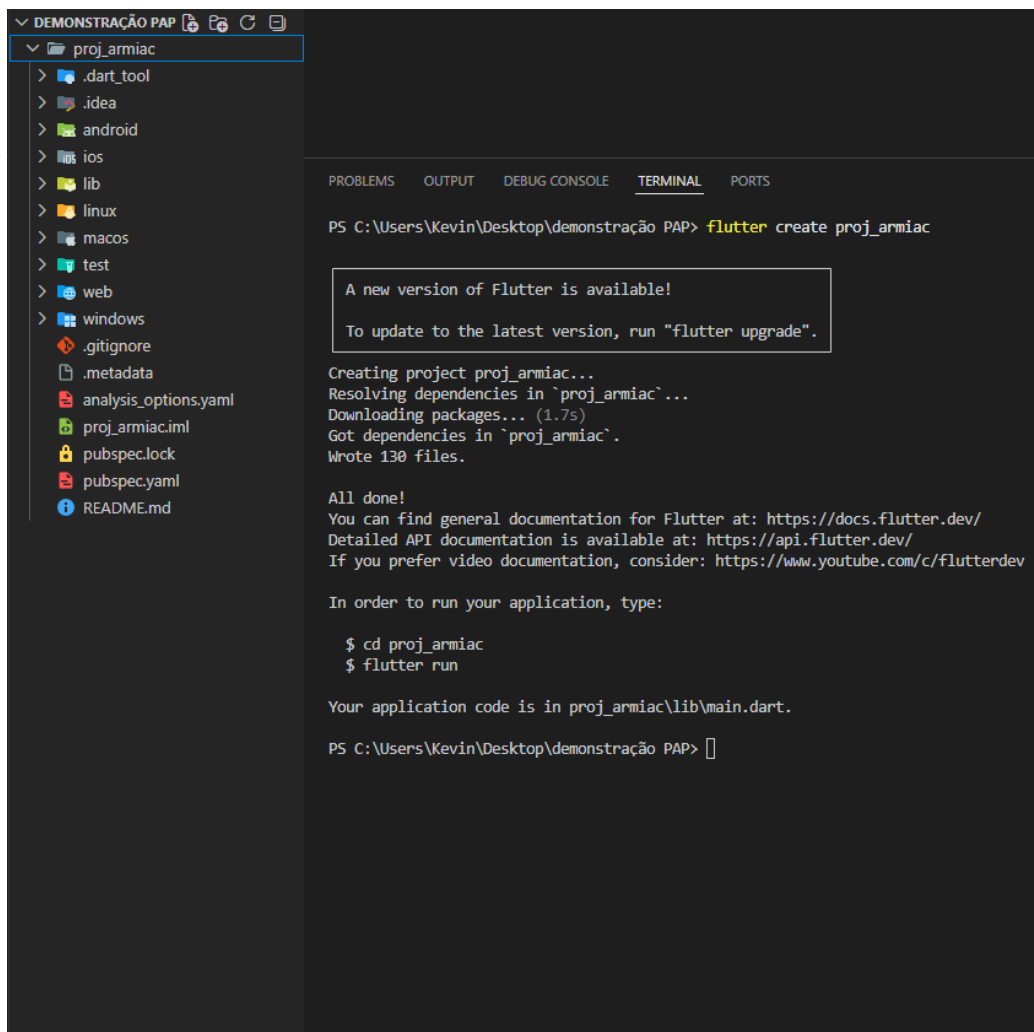


Figure 67: Creating the Flutter project

The ARMIAC application was developed and tested only for Android. Although Flutter can create applications for both Android and iOS, it was not possible to generate an iPhone version because a computer running macOS, such as a MacBook, would be required to compile the application. An iPhone would also be required to test installation, the interface, Wi-Fi connectivity, and command transmission to the ESP32. Since this equipment was not available, the final application was prepared only for Android.

4.4.2 - Dependency and Resource Management (**pubspec.yaml**)

The **pubspec.yaml** file is one of the most important files in a Flutter project because it defines:

- external dependencies (packages);
- the SDK version;
- project resources (assets), such as images; and
- basic application information, including its name and version.

In ARMIAC, **pubspec.yaml** was used to integrate essential libraries:

- **http** - used to make HTTP requests to the ESP32, including the `/ping` and `/action` endpoints;
- **package_info_plus** - used to obtain and display the application version in the settings area;
- **assets (images)** - used to load the project logo inside the application.

Correctly defining the dependencies and assets in **pubspec.yaml** was essential to ensure that the application worked correctly and that the logo image was loaded at the top of the application.

4.4.3 - Flutter Project Organization (Folder Structure)

During development of the ARMIAC application, the folders were organized with the aim of keeping the code cleaner and easier to understand.

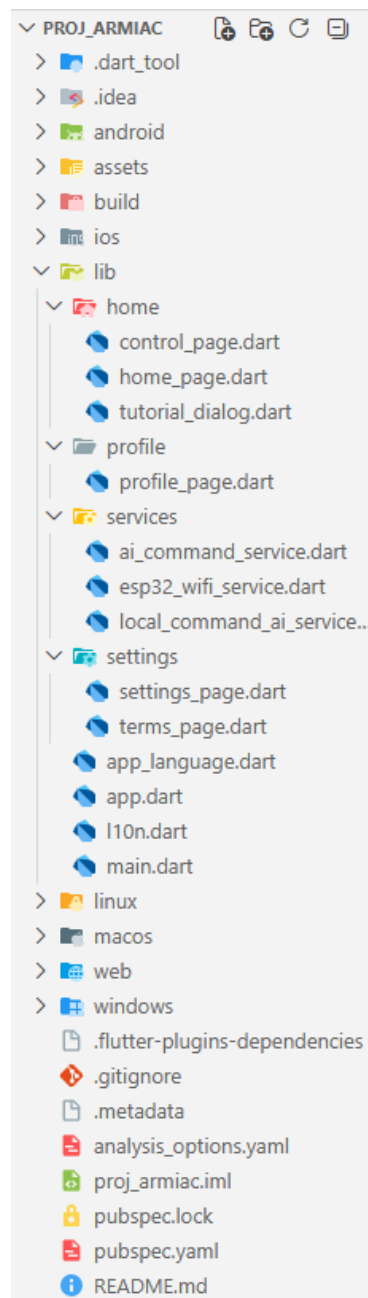


Figure 68: Flutter project folder structure

4.4.4 - Generating the Application APK

After development of the application was completed, an APK file was generated so that the ARMIAC application could be installed on Android devices.

The following command was used to create the APK:

```
flutter build apk --release
```

After the process was completed, the APK file was generated in:

```
build/app/outputs/flutter-apk/app-release.apk
```

This file can be installed on an Android phone, allowing the application to be used without having to run the project directly through Visual Studio Code.

The final ARMIAC application APK is available in the appendices, **section 7.3**, as stated in the original report.

4.5 - Project Visual Identity (ARMIAC Logo)

To give the project a visual identity, a logo bearing the ARMIAC name was developed. The logo was created using Canva, producing an image representing both the robotic arm and the name of the project.

The logo was later integrated into the Flutter application and is displayed at the top of the main interface and in the application icon.

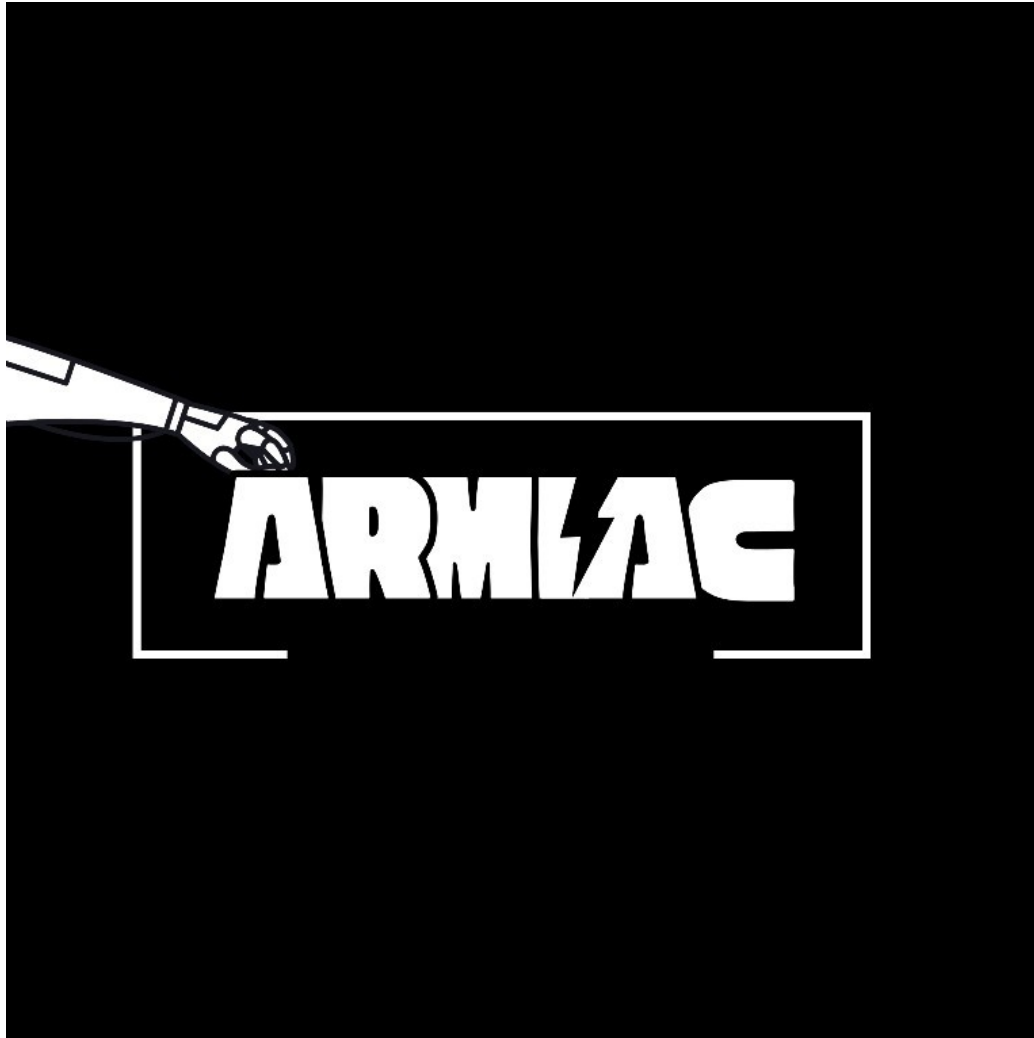


Figure 69: ARMIAC logo

4.6 - Difficulties Encountered During Development

Several technical difficulties arose during the development of the ARMIAC project and required research, testing, and adaptation of solutions in both the physical and software parts of the project.

One of the main difficulties was related to 3D printing. Some attempts resulted in model failures and printing errors, including deformation and adhesion problems on the print bed. This required several

prints to be repeated and adjustments to be made to the STL files and slicing settings. Although this increased development time, it provided a better understanding of the importance of correct model preparation and appropriate settings for the Elegoo Neptune 4 printer.

Another difficulty involved the finish of the parts. In the first version, an attempt was made to print the parts and then paint them, but the final result did not achieve the intended quality. The paint finish was not uniform and the visual appearance was unsatisfactory. For this reason, white PLA had to be purchased, an improved custom version of the parts was created, and the parts were printed again, producing a cleaner result that was better suited to the final prototype.

There were also difficulties with the system's power supply. A battery was initially tested because it appeared to be a more practical option and was expected to provide enough power for the servomotors and the remaining components. During testing, however, it became clear that the battery did not provide enough power to guarantee stable system operation.

During electronic testing, two PCA9685 modules were also damaged because an incorrect voltage was used. This delayed development because the damaged components had to be replaced and the electrical connections had to be reviewed more carefully, particularly the correct supply voltage for the modules and servomotors.

Another significant difficulty was installing and configuring Flutter in the development environment. During this process, errors occurred in which Flutter was not correctly recognized by the system because of problems with the **PATH** environment variable. The most frequent error was that the system could not locate the flutter command, preventing commands from being executed in the terminal. To solve this problem, the PATH had to be configured manually so that Flutter's bin folder was correctly added to the system variables.

In addition, although it was not exactly a technical problem, Dart had to be learned from the beginning because there was no previous experience with the language. It was necessary to study its syntax, Flutter project structure, and concepts such as widgets, navigation between pages, and package integration. This learning process was essential for developing a functional and stable application capable of controlling the prototype.

Despite these difficulties, they were all overcome through research, testing, and persistence, contributing to significant development of technical skills.

A demonstration is included under "**Demonstration of the project with battery vs. power supply**" in the appendices, section 7.3.

5 - Conclusion

This project made it possible to develop a functional robotic hand prototype controlled through a mobile application. ARMIAC made it possible to apply and connect several areas studied during the course, including programming, electronics, networking, application development, 3D printing, and technical problem solving.

The project was developed in stages, beginning with research and planning, followed by the creation and printing of the 3D parts, mechanical assembly, installation of the servomotors, connection to the PCA9685 module, programming of the ESP32, and development of the Flutter application. Several tests and adjustments were carried out throughout the process until a functional version of the prototype was obtained.

The mobile application, version **1.1.7**, allows ARMIAC to be controlled through Wi-Fi, buttons, quick commands, a movement panel, and voice commands. The application also includes Portuguese and

English support, a quick tutorial, a list of available commands, and a terms of service page, making it easier and more organized to use.

One of the most important parts of the project was the integration between the mobile application and the physical system. The application sends commands to the ESP32 using the **HTTP** protocol; the ESP32 communicates with the PCA9685 using **I2C**; and the PCA9685 controls the servomotors through **PWM** signals. This connection between software and hardware was essential for transforming commands from the application into real movements of the robotic hand.

Some difficulties arose during development, mainly relating to the electrical power supply, installation of the finger lines, calibration of the servomotors, and movement stability. The attempt to use a battery proved unstable and was later replaced by a **6V 10A DC power supply**, which provided safer and more appropriate operation of the servomotors.

Overall, the project achieved its main objective: creating a functional robotic hand that can be controlled remotely through different forms of interaction. ARMIAC also demonstrated educational potential and could be used as a learning platform for concepts related to robotics, programming, electronics, Wi-Fi communication, 3D printing, and mobile application development.

5.1 - Future Improvements

Although the project achieved its main objective, several improvements could be implemented in a future version of ARMIAC. One improvement would be the use of a more suitable battery capable of supplying sufficient power to the servomotors without causing instability. Pressure sensors could also be added to the fingers, allowing the arm to detect when it is holding an object.

Another improvement would be to increase movement precision through better servomotor calibration and a stronger mechanical structure. The mobile application could also be improved with new features such as custom movement profiles, command history, and an even more intuitive interface.

In the future, an iOS version of the application could also be developed if the equipment required to compile and test the application on Apple devices were available. ARMIAC could also be expanded to include Bluetooth control, sensor integration, or programmed automatic movements.

Future Improvement	Objective
More suitable battery	Make the project more portable
Pressure sensors in the fingers	Detect objects and grip force
Improved servo calibration	Smoother and more precise movements
Reinforced mechanical structure	Increase strength and durability
iOS version of the application	Enable use on iPhone
Custom movement profiles	Save gestures created by the user
Improved wire organization	Improve appearance and maintenance
Bluetooth control	Provide an alternative to Wi-Fi



Figure 70: Completed project

6 - Web References

- Arduino Documentation - <https://www.arduino.cc>
- Espressif ESP32 Documentation - <https://www.espressif.com>
- ESP32 Arduino Documentation - <https://docs.espressif.com>
- Adafruit PCA9685 Documentation - <https://learn.adafruit.com>
- Adafruit PWM Servo Driver Library - <https://github.com/adafruit/Adafruit-PWM-Servo-Driver-Library>
- PCA9685 Documentation - <https://www.nxp.com>
- Flutter Documentation - <https://docs.flutter.dev>
- Dart Documentation - <https://dart.dev>

- `http` package - <https://pub.dev/packages/http>
- `speech\_to\_text` package - https://pub.dev/packages/speech_to_text
- `flutter\_tts` package - https://pub.dev/packages/flutter_tts
- `shared\_preferences` package - https://pub.dev/packages/shared_preferences
- `package\_info\_plus` package - https://pub.dev/packages/package_info_plus
- LibreOffice - <https://www.libreoffice.org>
- Visual Studio Code Documentation - <https://code.visualstudio.com/docs>
- Android Studio Documentation - <https://developer.android.com/studio>
- Tinkercad - <https://www.tinkercad.com>
- Ultimaker Cura - <https://ultimaker.com/software/ultimaker-cura>
- Elegoo - <https://www.elegoo.com>
- Canva - <https://www.canva.com>
- MZ996R Servo - <https://www.aliexpress.com/s/wiki-ssr/article/mz996-servo>
- General Data Protection Regulation - <https://eur-lex.europa.eu>
- Portuguese National Data Protection Commission - <https://www.cnpd.pt>
- Base project used as inspiration - <https://www.thingiverse.com/thing:2269115>

7 - Appendices

7.1 - Source Code

The source-code appendix is reproduced from the original project report **without modifying the tested code**. For that reason, code identifiers, string literals, and original comments remain unchanged where they are part of the source files.

The appendix includes the ESP32 C++ code, Flutter/Dart application code, and the pubspec.yaml configuration used in the project.

Source-code note

The complete source-code appendix follows this translated report. The code is reproduced unchanged from the original project report so that the tested source files are not altered. Some comments and string literals therefore remain in Portuguese.

7 - Appendices

7.1 - Source Code

ESP32 Code (C++):

```
#include <Wire.h>
#include <WiFi.h>
#include <WebServer.h>
#include <Adafruit_PWMServoDriver.h>

Adafruit_PWMServoDriver pca = Adafruit_PWMServoDriver(0x40);
WebServer server(80);

const char* ssid = "ARMIAC";
const char* pass = "12345678";

#define SERVO_MIN 110
#define SERVO_MAX 500

#define CH_POLEGAR_DENTRO 0
#define CH_POLEGAR_FLEX 1
#define CH_INDICADOR 2
#define CH_MEDIO 3
#define CH_RINGPINKY 4
#define CH_ANTEBRACO 5

#define ANG_INDICADOR_ABERTO 180
#define ANG_INDICADOR_FECHADO 0
#define ANG_POLEGAR_DENTRO_ABERTO 0
#define ANG_POLEGAR_DENTRO_FECHADO 140

String lastAction = "open_hand";

// Converte o ângulo do servo para o valor de pulso usado pelo controlador PCA9685.
uint16_t anguloParaPulso(int angulo) {
    return map(angulo, 0, 180, SERVO_MIN, SERVO_MAX);
}

// Define o ângulo de um servo num determinado canal.
void setServoAng(int ch, int ang) {
    ang = constrain(ang, 0, 180);
    pca.setPWM(ch, 0, anguloParaPulso(ang));
}

// Devolve o ângulo invertido conforme o dedo esteja fechado ou aberto.
int angInvertido(int fechado) {
```

```

    return fechado ? 0 : 180;
}

// Controla o dedo indicador.
void setIndicador(int fechado) {
    setServoAng(CH_INDICADOR, fechado ? ANG_INDICADOR_FECHADO :
ANG_INDICADOR_ABERTO);
}

// Controla o dedo médio.
void setMedio(int fechado) {
    setServoAng(CH_MEDIO, angInvertido(fechado));
}

// Controla o anelar e o mindinho.
void setRingPinky(int fechado) {
    setServoAng(CH_RINGPINKY, angInvertido(fechado));
}

// Controla a flexão do polegar.
void setPolegarFlex(int fechado) {
    setServoAng(CH_POLEGAR_FLEX, angInvertido(fechado));
}

// Controla o movimento do polegar para dentro da mão.
void setPolegarDentro(int fechado) {
    int ang = fechado ? ANG_POLEGAR_DENTRO_FECHADO :
ANG_POLEGAR_DENTRO_ABERTO;
    setServoAng(CH_POLEGAR_DENTRO, ang);
}

// Abre todos os dedos da mão.
void actionOpenHand() {
    setIndicador(0);
    setMedio(0);
    setRingPinky(0);
    setPolegarFlex(0);
    setPolegarDentro(0);
}

// Fecha todos os dedos da mão.
void actionCloseHand() {
    setIndicador(1);
    setMedio(1);
    setRingPinky(1);
    setPolegarFlex(1);
    setPolegarDentro(1);
}

```

```

}

// Faz o gesto de paz.
void actionPeace() {
    setIndicador(0);
    setMedio(0);
    setRingPinky(1);
    setPolegarFlex(1);
    setPolegarDentro(1);
}

// Faz o gesto do número dois.
void actionTwo() {
    actionPeace();
}

// Faz o gesto de apontar.
void actionPoint() {
    setIndicador(0);
    setMedio(1);
    setRingPinky(1);
    setPolegarFlex(1);
    setPolegarDentro(1);
}

// Faz o gesto do número três.
void actionThree() {
    setIndicador(0);
    setMedio(0);
    setRingPinky(1);
    setPolegarFlex(0);
    setPolegarDentro(0);
}

// Faz o gesto do número quatro.
void actionFour() {
    setIndicador(0);
    setMedio(0);
    setRingPinky(0);
    setPolegarFlex(1);
    setPolegarDentro(1);
}

// Faz o gesto de polegar para cima.
void actionThumbsUp() {
    setIndicador(1);
    setMedio(1);

```

```

    setRingPinky(1);
    setPolegarFlex(0);
    setPolegarDentro(0);
}

// Faz uma aproximação do gesto OK.
void actionOk() {
    setIndicador(1);
    setMedio(0);
    setRingPinky(0);
    setPolegarFlex(1);
    setPolegarDentro(1);
}

// Roda o antebraço para uma posição com a palma para cima.
void actionPalmUp() {
    setServoAng(CH_ANTEBRACO, 35);
}

// Roda o antebraço para uma posição com a palma para baixo.
void actionPalmDown() {
    setServoAng(CH_ANTEBRACO, 145);
}

// Coloca o antebraço numa posição neutra.
void actionForearmNeutral() {
    setServoAng(CH_ANTEBRACO, 90);
}

// Faz um pequeno movimento de aceno com o antebraço.
void actionWave() {
    for (int repeticao = 0; repeticao < 2; repeticao++) {
        actionPalmUp();
        delay(250);
        actionPalmDown();
        delay(250);
    }

    actionForearmNeutral();
}

// Executa uma sequência de demonstração com vários gestos.
void actionDemo() {
    actionOpenHand();
    delay(500);

    actionCloseHand();
}

```

```

delay(500);

actionOpenHand();
delay(500);

actionPeace();
delay(700);

actionPoint();
delay(700);

actionThree();
delay(700);

actionThumbsUp();
delay(700);

actionOk();
delay(700);

actionWave();
delay(300);

    actionOpenHand();
}

// Fecha apenas o indicador.
void actionCloseIndex() {
    setIndicador(1);
}

// Fecha apenas o dedo médio.
void actionCloseMiddle() {
    setMedio(1);
}

// Fecha o anelar e o mindinho.
void actionCloseRingPinky() {
    setRingPinky(1);
}

// Fecha o polegar.
void actionCloseThumb() {
    setPolegarFlex(1);
    setPolegarDentro(1);
}

```

```

// Abre apenas o indicador.
void actionOpenIndex() {
    setIndicador(0);
}

// Abre apenas o dedo médio.
void actionOpenMiddle() {
    setMedio(0);
}

// Abre o anelar e o mindinho.
void actionOpenRingPinky() {
    setRingPinky(0);
}

// Abre o polegar.
void actionOpenThumb() {
    setPolegarFlex(0);
    setPolegarDentro(0);
}

// Responde ao pedido de teste da ligação.
void handlePing() {
    server.send(200, "text/plain", "OK");
}

// Envia o estado atual da mão em formato JSON.
void handleStatus() {
    String json = "{\"status\":\"ok\",\"lastAction\":\"" + lastAction + "\",\"ip\":\"" +
WiFi.softAPIP().toString() + "\"}";
    server.send(200, "application/json", json);
}

// Permite testar manualmente um servo através do navegador.
void handleCalibrate() {
    if (!server.hasArg("servo") || !server.hasArg("angle")) {
        server.send(400, "text/plain", "Use: /calibrate?servo=2&angle=90");
        return;
    }

    int servo = server.arg("servo").toInt();
    int angle = server.arg("angle").toInt();

    if (servo < 0 || servo > 15 || angle < 0 || angle > 180) {
        server.send(400, "text/plain", "servo deve ser 0-15 e angle deve ser 0-180");
        return;
    }
}

```

```

    setServoAng(servo, angle);
    lastAction = "calibrate servo " + String(servo) + " angle " + String(angle);

server.send(200, "text/plain", "OK");
}

// Recebe o nome da ação pela URL e executa o gesto correspondente.
void handleAction() {
    String a = server.arg("a");

    if (a == "close_hand") actionCloseHand();
    else if (a == "peace") actionPeace();
    else if (a == "two") actionTwo();
    else if (a == "point") actionPoint();
    else if (a == "three") actionThree();
    else if (a == "four") actionFour();
    else if (a == "thumbs_up") actionThumbsUp();
    else if (a == "ok") actionOk();
    else if (a == "wave") actionWave();
    else if (a == "palm_up") actionPalmUp();
    else if (a == "palm_down") actionPalmDown();
    else if (a == "forearm_neutral") actionForearmNeutral();
    else if (a == "demo") actionDemo();
    else if (a == "close_index") actionCloseIndex();
    else if (a == "close_middle") actionCloseMiddle();
    else if (a == "close_ringpinky") actionCloseRingPinky();
    else if (a == "close_thumb") actionCloseThumb();
    else if (a == "open_hand") actionOpenHand();
    else if (a == "open_index") actionOpenIndex();
    else if (a == "open_middle") actionOpenMiddle();
    else if (a == "open_ringpinky") actionOpenRingPinky();
    else if (a == "open_thumb") actionOpenThumb();
    else {
        server.send(400, "text/plain", "Acao invalida");
        return;
    }

    lastAction = a;
    server.send(200, "text/plain", "OK");
}

// Configura os servos, cria a rede Wi-Fi e inicia o servidor.
void setup() {
    Serial.begin(115200);

    Wire.begin(21, 22);

```

```

pca.begin();
pca.setPWMPFreq(50);
delay(10);

actionOpenHand();
actionForearmNeutral();

WiFi.mode(WIFI_AP);
WiFi.softAP(ssid, pass);

Serial.print("AP: ");
Serial.println(ssid);

Serial.print("IP: ");
Serial.println(WiFi.softAPIP());

server.on("/ping", handlePing);
server.on("/status", handleStatus);
server.on("/calibrate", handleCalibrate);
server.on("/action", handleAction);

server.begin();

Serial.println("READY");
}

// Mantém o servidor ativo para receber pedidos.
void loop() {
  server.handleClient();
}

```

Código Flutter(Dart):

Diretório – proj_armiac/lib/main.dart

```

import 'package:flutter/material.dart';
import 'app.dart';

/// Ponto de entrada da aplicação Flutter.
void main() {
  runApp(const MyApp());
}

```

Diretório – proj_armiac/lib/app.dart

```
import 'package:flutter/material.dart';
```

```
import 'app_language.dart';  
import 'home/home_page.dart';  
import 'l10n.dart';  
import 'settings/settings_page.dart';
```

/// Widget principal da aplicação. Fica responsável por montar o MaterialApp.

```
class MyApp extends StatelessWidget {  
  const MyApp({super.key});
```

```
  @override
```

/// Reconstrói a aplicação quando as definições globais mudam.

```
  Widget build(BuildContext context) {
```

```
    return AnimatedBuilder(  
      animation: appSettings,  
      builder: (context, _) {  
        return MaterialApp(  
          debugShowCheckedModeBanner: false,  
          title: 'ARMIAC',  
          locale: appSettings.language.locale,  
          theme: ThemeData(  
            colorScheme: ColorScheme.fromSeed(seedColor: Colors.deepPurple),  
            useMaterial3: true,  
          ),  
          home: const MyHomePage(),  
        );  
      },  
    );  
  }  
}
```

/// Página base com navegação entre o início e as configurações.

```
class MyHomePage extends StatefulWidget {  
  const MyHomePage({super.key});
```

```
  @override
```

```
  State<MyHomePage> createState() => _MyHomePageState();  
}
```

/// Guarda o separador selecionado e atualiza o ecrã quando o utilizador muda de página.

```
class _MyHomePageState extends State<MyHomePage> {  
  int _selectedIndex = 0;
```

/// Atualiza o índice da barra inferior.

```
  void _onItemTapped(int index) => setState(() => _selectedIndex = index);
```

@override

/// Reconstrói a aplicação quando as definições globais mudam.

```
Widget build(BuildContext context) {  
  return AnimatedBuilder(  
    animation: appSettings,  
    builder: (context, _) {  
      final l10n = L10n(appSettings.language);  
      final pages = const [HomePage(), SettingsPage()];  
  
      return Scaffold(  
        appBar: AppBar(  
          backgroundColor: Colors.black,  
          centerTitle: true,  
          toolbarHeight: 120,  
          title: SizedBox(  
            height: 200,  
            child: Image.asset(  
              'assets/images/Armiac_LOGO.png',  
              fit: BoxFit.contain,  
            ),  
          ),  
        ),  
        body: pages[_selectedIndex],  
        bottomNavigationBar: BottomNavigationBar(  
          currentIndex: _selectedIndex,  
          onTap: _onItemTapped,  
          backgroundColor: Colors.black,  
          selectedItemColor: Colors.white,  
          unselectedItemColor: Colors.grey,  
          items: [  
            BottomNavigationBarItem(  
              icon: const Icon(Icons.home),  
              label: l10n.home,  
            ),  
            BottomNavigationBarItem(  
              icon: const Icon(Icons.settings),  
              label: l10n.configShort,  
            ),  
          ],  
        ),  
      );  
    },  
  );  
}
```

Diretório – proj_armiac/lib/app_language.dart

```
import 'package:flutter/material.dart';

/// Idiomas suportados pela aplicação.
enum AppLanguage {
  portuguese,
  english,
}

/// Junta ao idioma os códigos usados pela interface, voz e leitura de texto.
extension AppLanguageInfo on AppLanguage {
  String get code => this == AppLanguage.portuguese ? 'pt_BR' : 'en_US';
  String get speechLocale => code;
  String get ttsLocale => this == AppLanguage.portuguese ? 'pt-PT' : 'en-US';
  Locale get locale => this == AppLanguage.portuguese ? const Locale('pt', 'PT') : const
  Locale('en', 'US');
  String get label => this == AppLanguage.portuguese ? 'Português' : 'English';
}

/// Controlador simples para guardar definições globais e avisar a interface quando
mudam.
class AppSettingsController extends ChangeNotifier {
  AppLanguage _language = AppLanguage.portuguese;

  AppLanguage get language => _language;

  /// Troca o idioma e obriga os widgets dependentes a atualizar.
  void setLanguage(AppLanguage value) {
    if (_language == value) return;
    _language = value;
    notifyListeners();
  }
}

/// Instância global usada pelas páginas para ler e alterar o idioma atual.
final appSettings = AppSettingsController();
```

Diretório – proj_armiac/lib/l10n.dart

```
import 'app_language.dart';

/// Centraliza todos os textos da aplicação em português e inglês.
class L10n {
  const L10n(this language)
```

```

final AppLanguage language;
bool get isPt => language == AppLanguage.portuguese;

String get home => isPt ? 'Início' : 'Home';
String get settings => isPt ? 'Configurações' : 'Settings';
String get configShort => isPt ? 'Config' : 'Settings';
String get appVersion => isPt ? 'Versão da app' : 'App version';
String get serviceTerms =>
  isPt ? 'Termos de serviço' : 'Terms of service';
String get appName => 'ARMIAC';
String get appSubtitle =>
  isPt ? 'Mão robótica inteligente' : 'Intelligent robotic hand';
String get languageLabel => isPt
  ? 'Idioma da app e comandos de voz'
  : 'App and voice command language';
String get portuguese => 'Português';
String get english => 'English';
String get languageHelp => isPt
  ? 'Ao escolher português, a app fala, escuta e entende comandos em português.'
  : 'When English is selected, the app speaks, listens, and understands English
commands.';
String get microphonePermission =>
  isPt ? 'Permissão do microfone' : 'Microphone permission';
String get microphonePermissionBody => isPt
  ? 'Ative a permissão do microfone nas definições do telemóvel para usar comandos
por voz.'
  : 'Enable microphone permission in your phone settings to use voice commands.';
String get armiacConnection =>
  isPt ? 'Ligação ao ARMIAC' : 'ARMIAC connection';
String get armiacConnectionBody => isPt
  ? 'Ligue o ARMIAC à corrente, conecte o telemóvel ao Wi-Fi “ARMIAC” e depois toque
em “Conectar ao ARMIAC”.'
  : 'Plug the ARMIAC into power, connect your phone to the “ARMIAC” Wi-Fi, and then
tap “Connect to ARMIAC”.';
String get connected => isPt ? 'ARMIAC Conectado' : 'ARMIAC Connected';
String get disconnected =>
  isPt ? 'ARMIAC Desconectado' : 'ARMIAC Disconnected';
String get connecting => isPt ? 'Conectando...' : 'Connecting...';
String get connectArmiac =>
  isPt ? 'Conectar ao ARMIAC' : 'Connect to ARMIAC';
String get stopVoice => isPt ? 'Parar voz' : 'Stop voice';
String get voiceControl => isPt ? 'Controlo por voz' : 'Voice control';
String get voice => isPt ? 'Voz' : 'Voice';
String get manualControl =>
  isPt ? 'Painel de movimentos' : 'Movement panel';
String get quickCommands => isPt ? 'Comandos rápidos' : 'Quick commands';

```

```

String get tutorial => isPt ? 'Tutorial rápido' : 'Quick tutorial';
String get tutorialTitle => isPt ? 'Como usar o ARMIAC' : 'How to use ARMIAC';
String get tutorialStep1Title => isPt ? '1. Ligar o ARMIAC' : '1. Power on ARMIAC';
String get tutorialStep1Body => isPt
    ? 'Ligue o ARMIAC à corrente para o deixar pronto a funcionar.'
    : 'Plug the ARMIAC into power so it is ready to work.';
String get tutorialStep2Title =>
    isPt ? '2. Conectar e usar' : '2. Connect and use';
String get tutorialStep2Body => isPt
    ? 'Conecte-se à rede Wi-Fi “ARMIAC” e toque no botão “Conectar ao ARMIAC” na
      app.'
    : 'Connect to the “ARMIAC” Wi-Fi network and tap “Connect to ARMIAC” in the app.';
String get previous => isPt ? 'Anterior' : 'Back';
String get next => isPt ? 'Seguinte' : 'Next';
String get finish => isPt ? 'Fechar' : 'Close';
String get connectedSnack =>
    isPt ? 'Conectado ao ARMIAC' : 'Connected to ARMIAC';
String get connectFirst => isPt
    ? 'Conecte-se primeiro ao Wi-Fi do ARMIAC.'
    : 'Connect to the ARMIAC Wi-Fi first.';
String get micUnavailable => isPt
    ? 'Microfone indisponível ou sem permissão.'
    : 'Microphone unavailable or permission denied.';
String get listening => isPt ? 'Estou a ouvir...' : 'Listening...';
String get speechNetworkError => isPt
    ? 'Internet desativada. Para usar voz em português sem internet, instale Português no
      reconhecimento de voz do telemóvel.'
    : 'Internet disabled. To use voice commands without internet, install the selected
      language in your phone voice recognition settings.';
String get speechLanguageError => isPt
    ? 'O português offline deste telemóvel está como Português (Brasil). A app vai tentar
      usar pt_BR.'
    : 'English is not available in this phone voice recognition. Enable or install English in
      the voice settings.';
String micError(String error) =>
    isPt ? 'Erro no microfone: $error' : 'Microphone error: $error';
String error(Object e) => isPt ? 'Erro: $e' : 'Error: $e';
String get actionFailed => isPt
    ? 'Falha ao enviar comando ao ARMIAC.'
    : 'Failed to send command to ARMIAC.';
String commandUnderstood(String spoken) =>
    isPt ? 'Comando entendido: $spoken' : 'Command understood: $spoken';
String get didNotUnderstand =>
    isPt ? 'Não entendi o comando' : 'I did not understand the command';
String get didNotUnderstandText => isPt
    ? 'Não entendi. Abra a ajuda para ver comandos disponíveis.'
    : 'I did not understand. Open help to see available commands.';

```

```

String get help => isPt ? 'Ajuda' : 'Help';
String get commandsHelp => isPt ? 'Comandos rápidos' : 'Quick commands';
String get commandsHelpIntro => isPt
    ? 'Pode falar de forma curta ou natural. A app tenta entender várias formas de dizer o
      mesmo comando.'
      : 'You can speak in a short or natural way. The app tries to understand many ways of
        saying the same command.';
List<String> get commandHelpItems => isPt
    ? [
        'Mão aberta: abrir mão, abre a mão, abre os dedos, estica os dedos, deixa a mão
        aberta',
        'Fechar mão: fechar mão, fecha a mão, feche os dedos, fecha os dedos, faz punho',
        'Número 2: número dois, dois dedos, mostra dois dedos',
        'Símbolo paz: paz, vitória, dois dedos, faz o V',
        'Apontar: apontar, aponta, só o indicador, número um',
        'Número 3: número três, três dedos, faz três, mostra três dedos',
        'Número 4: número quatro, quatro dedos, mostra quatro dedos',
        'Joinha: joinha, fixe, positivo, polegar para cima',
        'OK: ok, está bem, certo, perfeito',
        'Acenar: acenar, olá, adeus, cumprimenta',
        'Palma para cima: palma para cima, vira para cima',
        'Palma para baixo: palma para baixo, vira para baixo',
        'Antebraço ao centro: posição neutra, volta ao centro, antebraço neutro',
        'Demonstração: demonstração, mostra tudo, testa movimentos',
        'Indicador: abre o indicador, fecha o indicador',
        'Dedo médio: abre o dedo médio, fecha o dedo médio',
        'Anelar + mindinho: abre o anelar e o mindinho, fecha o anelar e o mindinho',
        'Polegar: abre o polegar, fecha o polegar',
    ]
    : [
        'Open hand: open hand, open the hand, open fingers, stretch fingers',
        'Close hand: close hand, close the fingers, make a fist, squeeze hand',
        'Number 2: number two, two fingers, show two fingers',
        'Peace sign: peace, victory, two fingers, make a V',
        'Point: point, only index, number one, point forward',
        'Number 3: number three, three fingers, show three',
        'Number 4: number four, four fingers, show four fingers',
        'Thumbs up: thumbs up, like, positive sign',
        'OK: ok, okay, perfect',
        'Wave: wave, hello, bye, greet',
        'Palm up: palm up, face up, rotate up',
        'Palm down: palm down, face down, rotate down',
        'Forearm center: neutral forearm, return to center',
        'Demo: demo, demonstration, show everything',
        'Index finger: open index, close index',
        'Middle finger: open middle finger, close middle finger',
        'Ring + pinky: open ring and pinky, close ring and pinky',
    ]

```

```

    'Thumb: open thumb, close thumb',
];

String get open => isPt ? 'Abrir' : 'Open';
String get close => isPt ? 'Fechar' : 'Close';
String get peace => isPt ? 'Paz' : 'Peace';
String get point => isPt ? 'Apontar' : 'Point';
String get thumbsUp => isPt ? 'Joinha' : 'Thumbs up';
String get wave => isPt ? 'Acenar' : 'Wave';
String get palmUpShort => isPt ? 'Palma cima' : 'Palm up';
String get palmDownShort => isPt ? 'Palma baixo' : 'Palm down';
String get handControl =>
    isPt ? 'Painel de movimentos' : 'Movement panel';
String get connectedToArmiac =>
    isPt ? 'Ligado ao ARMIAC' : 'Connected to ARMIAC';
String get notConnectedToArmiac =>
    isPt ? 'Não ligado ao ARMIAC' : 'Not connected to ARMIAC';
String get status => isPt ? 'Estado' : 'Status';
String get forearm => isPt ? 'Antebraço' : 'Forearm';
String get openCloseHand =>
    isPt ? 'Abrir / fechar mão' : 'Open / close hand';
String get closeHand => isPt ? 'Fechar mão' : 'Close hand';
String get peaceSign => isPt ? 'Símbolo paz' : 'Peace sign';
String get numberTwo => isPt ? 'Número 2' : 'Number 2';
String get numberThree => isPt ? 'Número 3' : 'Number 3';
String get numberFour => isPt ? 'Número 4' : 'Number 4';
String get okSign => 'OK';
String get palmUp => isPt ? 'Palma para cima' : 'Palm up';
String get palmDown => isPt ? 'Palma para baixo' : 'Palm down';
String get forearmNeutral =>
    isPt ? 'Antebraço ao centro' : 'Forearm center';
String get automaticDemo =>
    isPt ? 'Demonstração automática' : 'Automatic demo';
String get indexFinger => isPt ? 'Dedo indicador' : 'Index finger';
String get middleFinger => isPt ? 'Dedo médio' : 'Middle finger';
String get ringPinky =>
    isPt ? 'Dedo anelar + mindinho' : 'Ring + pinky fingers';
String get thumb => isPt ? 'Dedão' : 'Thumb';

String get openHandState => isPt ? 'Mão aberta' : 'Open hand';
String get closedFistState => isPt ? 'Punho fechado' : 'Closed fist';
String get demoState => isPt ? 'Demonstração' : 'Demo';
String get palmUpState => isPt ? 'Palma para cima' : 'Palm up';
String get palmDownState => isPt ? 'Palma para baixo' : 'Palm down';
String get forearmCenterState =>
    isPt ? 'Antebraço ao centro' : 'Forearm center';
String get indexClosed => isPt ? 'Indicador fechado' : 'Index closed';

```

```

String get indexOpen => isPt ? 'Indicador aberto' : 'Index open';
String get middleClosed =>
    isPt ? 'Dedo médio fechado' : 'Middle finger closed';
String get middleOpen => isPt ? 'Dedo médio aberto' : 'Middle finger open';
String get ringPinkyClosed =>
    isPt ? 'Anelar + mindinho fechados' : 'Ring + pinky closed';
String get ringPinkyOpen =>
    isPt ? 'Anelar + mindinho abertos' : 'Ring + pinky open';
String get thumbClosed => isPt ? 'Polegar fechado' : 'Thumb closed';
String get thumbOpen => isPt ? 'Polegar aberto' : 'Thumb open';

/// Devolve a frase falada depois de um comando ser executado.
String speechFor(
    String command, {
    bool indexClosed = false,
    bool middleClosed = false,
    bool ringPinkyClosed = false,
    bool thumbClosed = false,
}) {
    switch (command) {
        case 'open_hand':
            return isPt ? 'vou abrir a mão' : 'opening the hand';
        case 'close_hand':
            return isPt ? 'vou fechar a mão' : 'closing the hand';
        case 'peace':
            return isPt ? 'vou fazer o gesto de paz' : 'making the peace sign';
        case 'two':
            return isPt ? 'vou mostrar o número dois' : 'showing number two';
        case 'point':
            return isPt
                ? 'vou apontar com o indicador'
                : 'pointing with the index finger';
        case 'three':
            return isPt ? 'vou mostrar o número três' : 'showing number three';
        case 'four':
            return isPt ? 'vou mostrar o número quatro' : 'showing number four';
        case 'thumbs_up':
            return isPt ? 'vou fazer joinha' : 'making a thumbs up';
        case 'ok':
            return isPt ? 'vou fazer o gesto ok' : 'making the OK sign';
        case 'wave':
            return isPt ? 'vou acenar' : 'waving';
        case 'palm_up':
            return isPt ? 'vou virar a palma para cima' : 'turning the palm up';
        case 'palm_down':
            return isPt ? 'vou virar a palma para baixo' : 'turning the palm down';
        case 'forearm_neutral':
    
```

```

return isPt
    ? 'vou pôr o antebraço ao centro'
    : 'moving the forearm to the center';
case 'demo':
    return isPt ? 'vou fazer a demonstração' : 'starting the demonstration';
case 'open_index':
    return isPt ? 'vou abrir o indicador' : 'opening the index finger';
case 'close_index':
    return isPt ? 'vou fechar o indicador' : 'closing the index finger';
case 'open_middle':
    return isPt ? 'vou abrir o dedo médio' : 'opening the middle finger';
case 'close_middle':
    return isPt ? 'vou fechar o dedo médio' : 'closing the middle finger';
case 'open_ringpinky':
    return isPt
        ? 'vou abrir o anelar e o mindinho'
        : 'opening the ring and pinky fingers';
case 'close_ringpinky':
    return isPt
        ? 'vou fechar o anelar e o mindinho'
        : 'closing the ring and pinky fingers';
case 'open_thumb':
    return isPt ? 'vou abrir o polegar' : 'opening the thumb';
case 'close_thumb':
    return isPt ? 'vou fechar o polegar' : 'closing the thumb';
case 'toggle_index':
    return isPt
        ? (indexClosed ? 'vou abrir o indicador' : 'vou fechar o indicador')
        : (indexClosed
            ? 'opening the index finger'
            : 'closing the index finger');
case 'toggle_middle':
    return isPt
        ? (middleClosed
            ? 'vou abrir o dedo médio'
            : 'vou fechar o dedo médio')
        : (middleClosed
            ? 'opening the middle finger'
            : 'closing the middle finger');
case 'toggle_ring_pinky':
    return isPt
        ? (ringPinkyClosed
            ? 'vou abrir o anelar e o mindinho'
            : 'vou fechar o anelar e o mindinho')
        : (ringPinkyClosed
            ? 'opening the ring and pinky fingers'
            : 'closing the ring and pinky fingers');

```

```

    case 'toggle_thumb':
        return isPt
            ? (thumbClosed ? 'vou abrir o polegar' : 'vou fechar o polegar')
            : (thumbClosed ? 'opening the thumb' : 'closing the thumb');
    }
    return "";
}

/// Textos usados na página de termos de serviço.
String get termsTitle => isPt ? 'Termos de Serviço' : 'Terms of Service';
String get termsHeader =>
    isPt ? 'TERMOS DE SERVIÇO – ARMIAC' : 'TERMS OF SERVICE – ARMIAC';
String get lastUpdate => isPt
    ? 'Última atualização: Maio de 2026'
    : 'Last updated: May 2026';
List<String> get termsParagraphs => isPt
    ? [
        'ARMIAC é um projeto independente desenvolvido por Quévin Leonardo Aguiar Tavares em Portugal.',
        'A aplicação permite aos utilizadores controlar funcionalidades digitais e comandos ligados à mão robótica ARMIAC.',
        'O utilizador deve usar a aplicação de forma responsável, segura e de acordo com a lei.',
        'O nome, logótipo, design, código e identidade visual ARMIAC pertencem ao proprietário do projeto.',
        'A ARMIAC respeita a privacidade do utilizador e procura processar dados pessoais de forma segura, em conformidade com o RGPD quando aplicável.',
        'Use os comandos da mão robótica com cuidado. O utilizador é responsável por testar o sistema num ambiente seguro.',
        'Contacto: kevinleonardomail@gmail.com | Website: https://kevinleonardohub.github.io/portfolio',
    ]
    : [
        'ARMIAC is an independent project developed by Quévin Leonardo Aguiar Tavares in Portugal.',
        'The application allows users to control digital features and commands linked to the ARMIAC robotic hand.',
        'The user must use the application responsibly, safely, and in accordance with the law.',
        'The ARMIAC name, logo, design, code, and visual identity belong to the project owner.',
        'ARMIAC respects user privacy and aims to process personal data securely, in line with GDPR when applicable.',
        'Use robotic hand commands carefully. The user is responsible for testing the system in a safe environment.',
        'Contact: kevinleonardomail@gmail.com | Website: https://kevinleonardohub.github.io/portfolio',
    ]

```

```
];
```

```
String get speechServerDisconnectedError => isPt  
  ? 'O motor de voz desligou neste telemóvel. A app vai tentar sem forçar o modo local.  
  Toque novamente em Controlo por voz.'  
  : 'The voice engine disconnected on this phone. The app will try without forcing local  
  mode. Tap Voice control again.';  
}
```

Diretório – proj_armiac/lib/home/control_page.dart

```
import 'package:flutter/material.dart';  
  
import '../app_language.dart';  
import '../l10n.dart';  
import '../services/esp32_wifi_service.dart';  
  
/// Página de controlo manual da mão robótica.  
class ControlPage extends StatefulWidget {  
  const ControlPage({super.key});  
  
  @override  
  State<ControlPage> createState() => _ControlPageState();  
}  
  
/// Mantém o estado visual dos dedos e envia comandos para o ESP32.  
class _ControlPageState extends State<ControlPage> {  
  bool handClosed = false;  
  bool peaceMode = false;  
  bool indexClosed = false;  
  bool middleClosed = false;  
  bool ringPinkyClosed = false;  
  bool thumbClosed = false;  
  String lastGestureKey = 'open_hand';  
  String forearmKey = 'forearm_neutral';  
  bool _demoRunning = false;  
  
  L10n get _l10n => L10n(appSettings.language);  
  
  /// Converte a chave interna de um gesto no texto apresentado ao utilizador.  
  String _gestureLabel(String key) {  
    final l10n = _l10n;  
    switch (key) {  
      case 'open_hand':  
        return l10n.openHandState;  
      case 'close_hand':
```

```

        return l10n.closedFistState;
    case 'peace':
        return l10n.peaceSign;
    case 'two':
        return l10n.numberTwo;
    case 'point':
        return l10n.point;
    case 'three':
        return l10n.numberThree;
    case 'four':
        return l10n.numberFour;
    case 'thumbs_up':
        return l10n.thumbsUp;
    case 'ok':
        return l10n.okSign;
    case 'wave':
        return l10n.wave;
    case 'demo':
        return l10n.demoState;
    case 'index_closed':
        return l10n.indexClosed;
    case 'index_open':
        return l10n.indexOpen;
    case 'middle_closed':
        return l10n.middleClosed;
    case 'middle_open':
        return l10n.middleOpen;
    case 'ring_pinky_closed':
        return l10n.ringPinkyClosed;
    case 'ring_pinky_open':
        return l10n.ringPinkyOpen;
    case 'thumb_closed':
        return l10n.thumbClosed;
    case 'thumb_open':
        return l10n.thumbOpen;
    }
    return key;
}

```

/// Converte a posição do antebraço num texto legível.

```

String_forearmLabel(String key) {
    final l10n = _l10n;
    switch (key) {
        case 'palm_up':
            return l10n.palmUpState;
    case 'palm_down':
        return l10n.palmDownState;
    }
}

```

```

    default:
        return l10n.forearmCenterState;
    }
}

/// Envia uma ação para o ESP32 e devolve se o pedido correu bem.
Future<bool> _send(String action) async {
    if (!Esp32WifiService.instance.isConnected.value) {
        ScaffoldMessenger.of(context).showSnackBar(SnackBar(content:
Text(_l10n.connectFirst)));
        return false;
    }

    try {
        await Esp32WifiService.instance.action(action);
        return true;
    } catch (e) {
        if (!mounted) return false;
        ScaffoldMessenger.of(context).showSnackBar(SnackBar(content:
Text(_l10n.actionFailed)));
        return false;
    }
}

/// Atualiza o estado local para representar a mão aberta.
void _setOpenHand() {
    handClosed = false;
    peaceMode = false;
    indexClosed = false;
    middleClosed = false;
    ringPinkyClosed = false;
    thumbClosed = false;
    lastGestureKey = 'open_hand';
}

/// Atualiza o estado local para representar a mão fechada.
void _setClosedHand() {
    handClosed = true;
    peaceMode = false;
    indexClosed = true;
    middleClosed = true;
    ringPinkyClosed = true;
    thumbClosed = true;
    lastGestureKey = 'close_hand';
}

/// Sincroniza a interface com o gesto que acabou de ser executado.

```

```

void _applyActionState(String action) {
    switch (action) {
        case 'open_hand':
            _setOpenHand();
            break;
        case 'close_hand':
            _setClosedHand();
            break;
        case 'peace':
            handClosed = false;
            peaceMode = true;
            indexClosed = false;
            middleClosed = false;
            ringPinkyClosed = true;
            thumbClosed = true;
            lastGestureKey = 'peace';
            break;
        case 'two':
            handClosed = false;
            peaceMode = false;
            indexClosed = false;
            middleClosed = false;
            ringPinkyClosed = true;
            thumbClosed = true;
            lastGestureKey = 'two';
            break;
        case 'point':
            handClosed = false;
            peaceMode = false;
            indexClosed = false;
            middleClosed = true;
            ringPinkyClosed = true;
            thumbClosed = true;
            lastGestureKey = 'point';
            break;
        case 'three':
            handClosed = false;
            peaceMode = false;
            indexClosed = false;
            middleClosed = false;
            ringPinkyClosed = true;
            thumbClosed = false;
            lastGestureKey = 'three';
            break;
        case 'four':
            handClosed = false;
            peaceMode = false;
    }
}

```

```

    indexClosed = false;
    middleClosed = false;
    ringPinkyClosed = false;
    thumbClosed = true;
    lastGestureKey = 'four';
    break;
case 'thumbs_up':
    handClosed = false;
    peaceMode = false;
    indexClosed = true;
    middleClosed = true;
    ringPinkyClosed = true;
    thumbClosed = false;
    lastGestureKey = 'thumbs_up';
    break;
case 'ok':
    handClosed = false;
    peaceMode = false;
    indexClosed = true;
    middleClosed = false;
    ringPinkyClosed = false;
    thumbClosed = false;
    lastGestureKey = 'ok';
    break;
case 'wave':
    handClosed = false;
    peaceMode = false;
    indexClosed = false;
    middleClosed = false;
    ringPinkyClosed = false;
    thumbClosed = false;
    lastGestureKey = 'wave';
    break;
case 'palm_up':
case 'palm_down':
case 'forearm_neutral':
    forearmKey = action;
    break;
case 'demo':
    lastGestureKey = 'demo';
    break;
}
}

```

```

/// Envia o comando e só atualiza o ecrã se o ESP32 responder corretamente.
Future<bool> _sendActionAndUpdate(String action) async {
    final ok = await _send(action);

```

```

if (!ok || !mounted) return false;
setState(() => _applyActionState(action));
return true;
}

```

/// Executa um gesto simples, bloqueando ações durante a demonstração.

```

Future<void> _gesture(String action) async {
  if (!_demoRunning) return;

```

```

  if (action == 'demo') {
    await _runDemo();
    return;
  }

```

```

  if (action == 'wave') {
    final opened = await _sendActionAndUpdate('open_hand');
    if (!opened) return;
    await Future<void>.delayed(const Duration(milliseconds: 180));
    await _sendActionAndUpdate('wave');
    return;
  }

```

```

  final sent = await _sendActionAndUpdate(action);
  if (!sent || !mounted) return;

```

```

  final isHandGesture = ![
    'palm_up',
    'palm_down',
    'forearm_neutral',
  ].contains(action);

```

```

  if (isHandGesture && forearmKey != 'forearm_neutral') {
    await Future<void>.delayed(const Duration(milliseconds: 120));
    await _send(forearmKey);
  }
}

```

/// Corre a sequência de demonstração e mantém o estado final coerente.

```

Future<void> _runDemo() async {
  if (!_demoRunning) return;
  setState(() => _demoRunning = true);

```

```

  final sequence = [
    'open_hand',
    'palm_up',
    'peace',
    'two',

```

```

    'palm_down',
    'point',
    'forearm_neutral',
    'three',
    'four',
    'palm_up',
    'thumbs_up',
    'palm_down',
    'ok',
    'forearm_neutral',

    'open_hand',
    'wave',
    'forearm_neutral',
    'open_hand',
];

for (final action in sequence) {
  if (!mounted) return;
  final ok = await _sendActionAndUpdate(action);
  if (!ok) break;
  await Future<void>.delayed(const Duration(milliseconds: 450));
}

if (mounted) {
  setState(() {
    _demoRunning = false;
    lastGestureKey = 'demo';
  });
}
}

/// Alterna entre mão aberta e mão fechada.
Future<void> _toggleHand() async {
  await _gesture(handClosed ? 'open_hand' : 'close_hand');
}

/// Alterna o estado do indicador.
Future<void> _toggleIndex() async {
  final ok = await _send(indexClosed ? 'open_index' : 'close_index');
  if (!ok || !mounted) return;
  setState(() {
    indexClosed = !indexClosed;
    handClosed = false;
    peaceMode = false;
    lastGestureKey = indexClosed ? 'index_closed' : 'index_open';
  });
}

```

```

if (forearmKey != 'forearm_neutral') {
  await Future<void>.delayed(const Duration(milliseconds: 120));
  await _send(forearmKey);
}
}

```

/// Alterna o estado do dedo médio.

```

Future<void> _toggleMiddle() async {
  final ok = await _send(middleClosed ? 'open_middle' : 'close_middle');
  if (!ok || !mounted) return;
  setState(() {
    middleClosed = !middleClosed;
    handClosed = false;
    peaceMode = false;
    lastGestureKey = middleClosed ? 'middle_closed' : 'middle_open';
  });
}

```

```

if (forearmKey != 'forearm_neutral') {
  await Future<void>.delayed(const Duration(milliseconds: 120));
  await _send(forearmKey);
}
}

```

/// Alterna o estado do anelar e do mindinho.

```

Future<void> _toggleRingPinky() async {
  final ok = await _send(ringPinkyClosed ? 'open_ringpinky' : 'close_ringpinky');
  if (!ok || !mounted) return;
  setState(() {
    ringPinkyClosed = !ringPinkyClosed;
    handClosed = false;
    peaceMode = false;
    lastGestureKey = ringPinkyClosed ? 'ring_pinky_closed' : 'ring_pinky_open';
  });
}

```

```

if (forearmKey != 'forearm_neutral') {
  await Future<void>.delayed(const Duration(milliseconds: 120));
  await _send(forearmKey);
}
}

```

/// Alterna o estado do polegar.

```

Future<void> _toggleThumb() async {
  final ok = await _send(thumbClosed ? 'open_thumb' : 'close_thumb');
  if (!ok || !mounted) return;
  setState(() {
    thumbClosed = !thumbClosed;
  });
}

```

```

handClosed = false;
peaceMode = false;
lastGestureKey = thumbClosed ? 'thumb_closed' : 'thumb_open';
});

```

```

if (forearmKey != 'forearm_neutral') {
  await Future<void>.delayed(const Duration(milliseconds: 120));
  await _send(forearmKey);
}
}

```

/// Cria um botão reutilizável para comandos rápidos.

```

Widget _actionButton({required String label, required IconData icon, required
VoidCallback? onPressed, Color? color}) {
  return SizedBox(
    width: double.infinity,
    child: ElevatedButton.icon(
      onPressed: onPressed,
      icon: Icon(icon),
      style: ElevatedButton.styleFrom(backgroundColor: color),
      label: Text(label),
    ),
  );
}

```

/// Cria um botão que mostra se uma parte da mão está aberta ou fechada.

```

Widget _toggleButton({required String label, required bool state, required VoidCallback?
onPressed}) {
  return SizedBox(
    width: double.infinity,
    child: ElevatedButton(
      onPressed: onPressed,
      style: ElevatedButton.styleFrom(backgroundColor: state ? Colors.green : null),
      child: Text(state ? '$label ': label),
    ),
  );
}

```

@override

```

Widget build(BuildContext context) {
  return AnimatedBuilder(
    animation: appSettings,
    builder: (context, _) {
      final l10n = _l10n;
      return Scaffold(
        appBar: AppBar(title: Text(l10n.handControl)),
        body: ValueListenableBuilder<bool>(

```

```

valueListenable: Esp32WifiService.instance.isConnected,
builder: (context, connected, _) => SingleChildScrollView(
  padding: const EdgeInsets.all(16),
  child: Column(
    children: [
      Container(
        width: double.infinity,
        padding: const EdgeInsets.all(12),
        decoration: BoxDecoration(
          border: Border.all(color: connected ? Colors.green : Colors.red),
          borderRadius: BorderRadius.circular(12),
        ),
        child: Row(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            Icon(connected ? Icons.wifi : Icons.wifi_off),
            const SizedBox(width: 8),
            Text(
              connected ? I10n.connectedToArmiac : I10n.notConnectedToArmiac,
              style: const TextStyle(fontWeight: FontWeight.bold),
            ),
          ],
        ),
      ),
      const SizedBox(height: 12),
      Text('${I10n.status}: ${_gestureLabel(lastGestureKey)}', style: const
TextStyle(fontWeight: FontWeight.bold)),
      const SizedBox(height: 4),
      Text('${I10n.forearm}: ${_forearmLabel(forearmKey)}', style: const
TextStyle(fontWeight: FontWeight.bold)),
      const SizedBox(height: 14),
      _actionButton(label: I10n.openCloseHand, icon: Icons.pan_tool_alt, onPressed:
connected && !_demoRunning ? _toggleHand : null),
      const SizedBox(height: 10),
      _actionButton(label: I10n.peaceSign, icon: Icons.gesture, onPressed:
connected && !_demoRunning ? () => _gesture('peace') : null),
      const SizedBox(height: 10),
      _actionButton(label: I10n.numberTwo, icon: Icons.looks_two, onPressed:
connected && !_demoRunning ? () => _gesture('two') : null),
      const SizedBox(height: 10),
      _actionButton(label: I10n.point, icon: Icons.touch_app, onPressed: connected
&& !_demoRunning ? () => _gesture('point') : null),
      const SizedBox(height: 10),
      _actionButton(label: I10n.numberThree, icon: Icons.looks_3, onPressed:
connected && !_demoRunning ? () => _gesture('three') : null),
      const SizedBox(height: 10),

```

```

        _actionButton(label: l10n.numberFour, icon: Icons.looks_4, onPressed:
connected && !_demoRunning ? () => _gesture('four') : null),
        const SizedBox(height: 10),
        _actionButton(label: l10n.thumbsUp, icon: Icons.thumb_up, onPressed:
connected && !_demoRunning ? () => _gesture('thumbs_up') : null),
        const SizedBox(height: 10),
        _actionButton(label: l10n.okSign, icon: Icons.check_circle_outline, onPressed:
connected && !_demoRunning ? () => _gesture('ok') : null),
        const SizedBox(height: 10),
        _actionButton(label: l10n.wave, icon: Icons.waving_hand, onPressed:
connected && !_demoRunning ? () => _gesture('wave') : null),
        const SizedBox(height: 10),
        _actionButton(label: l10n.palmUp, icon: Icons.keyboard_arrow_up, onPressed:
connected && !_demoRunning ? () => _gesture('palm_up') : null),
        const SizedBox(height: 10),
        _actionButton(label: l10n.palmDown, icon: Icons.keyboard_arrow_down,
onPressed: connected && !_demoRunning ? () => _gesture('palm_down') : null),
        const SizedBox(height: 10),
        _actionButton(label: l10n.forearmNeutral, icon: Icons.swap_vert, onPressed:
connected && !_demoRunning ? () => _gesture('forearm_neutral') : null),
        const SizedBox(height: 10),
        _actionButton(label: l10n.automaticDemo, icon: Icons.play_arrow, onPressed:
connected && !_demoRunning ? () => _gesture('demo') : null),
        const SizedBox(height: 20),
        _toggleButton(label: l10n.indexFinger, state: indexClosed, onPressed:
connected && !_demoRunning ? _toggleIndex : null),
        const SizedBox(height: 10),
        _toggleButton(label: l10n.middleFinger, state: middleClosed, onPressed:
connected && !_demoRunning ? _toggleMiddle : null),
        const SizedBox(height: 10),
        _toggleButton(label: l10n.ringPinky, state: ringPinkyClosed, onPressed:
connected && !_demoRunning ? _toggleRingPinky : null),
        const SizedBox(height: 10),
        _toggleButton(label: l10n.thumb, state: thumbClosed, onPressed: connected
&& !_demoRunning ? _toggleThumb : null),
    ],
  ),
),
),
),
);
},
);
}
}
}

```

Diretório – proj_armiac/lib/home/home_page.dart

```
import 'package:flutter/material.dart';
import 'package:flutter_tts/flutter_tts.dart';
import 'package:speech_to_text/speech_to_text.dart' as stt;
import 'package:shared_preferences/shared_preferences.dart';

import '../app_language.dart';
import '../l10n.dart';
import '../services/esp32_wifi_service.dart';
import '../services/local_command_ai_service.dart';
import 'control_page.dart';
import 'tutorial_dialog.dart';

/// Página inicial, com ligação ao ARMIAC, voz, IA local e atalhos principais.
class HomePage extends StatefulWidget {
  const HomePage({super.key});

  @override
  State<HomePage> createState() => _HomePageState();
}

/// Comandos internos que a aplicação consegue transformar em ações para o ESP32.
enum _ArmiacCommand {
  openHand,
  closeHand,
  peace,
  two,
  point,
  three,
  four,
  thumbsUp,
  ok,
  wave,
  palmUp,
  palmDown,
  forearmNeutral,
  demo,
  openIndex,
  closeIndex,
  openMiddle,
  closeMiddle,
  openRingPinky,
  closeRingPinky,
  openThumb,
  closeThumb,
}
```

```

class _HomePageState extends State<HomePage> {
  bool _loading = false;
  bool _listening = false;
  bool _demoRunning = false;
  String _lastVoiceText = "";
  String _aiFeedback = "";

  stt.SpeechToText _speech = stt.SpeechToText();
  final FlutterTts _tts = FlutterTts();

  bool handClosed = false;
  bool peaceMode = false;
  bool indexClosed = false;
  bool middleClosed = false;
  bool ringPinkyClosed = false;
  bool thumbClosed = false;
  String forearmKey = 'forearm_neutral';

  @override
  /// Prepara a voz e mostra o tutorial inicial quando necessário.
  void initState() {
    super.initState();
    _configureTts();
    WidgetsBinding.instance.addPostFrameCallback((_) {
      _openTutorialOnce();
    });
  }

  @override
  /// Liberta recursos de voz antes de sair da página.
  void dispose() {
    _speech.stop();
    _tts.stop();
    super.dispose();
  }

  L10n get _l10n => L10n(appSettings.language);

  /// Configura a voz que responde ao utilizador.
  Future<void> _configureTts() async {
    await _tts.setLanguage(appSettings.language.ttsLocale);
    await _tts.setSpeechRate(0.9);
    await _tts.setPitch(1.0);
    await _tts.awaitSpeakCompletion(true);
  }
}

```

```

/// Fala uma resposta curta sem deixar falas antigas acumuladas.
Future<void> _speakFast(String text) async {
  await _tts.stop();
  await _configureTts();
  await _tts.speak(text);
}

/// Mostra o tutorial apenas na primeira utilização da aplicação.
Future<void> _openTutorialOnce() async {
  final prefs = await SharedPreferences.getInstance();
  final alreadyShown = prefs.getBool('tutorial_shown_v1_1_4') ?? false;

  if (!mounted || alreadyShown) return;

  await prefs.setBool('tutorial_shown_v1_1_4', true);
  _openTutorial();
}

/// Abre manualmente o tutorial.
void _openTutorial() {
  showDialog<void>(
    context: context,
    builder: (_) => const TutorialDialog(),
  );
}

/// Testa a comunicação com o ESP32 e atualiza o estado de ligação.
Future<void> _connect() async {
  setState(() => _loading = true);
  try {
    await Esp32WifiService.instance.connect();
    if (!mounted) return;
    ScaffoldMessenger.of(
      context,
    ).showSnackBar(SnackBar(content: Text(_l10n.connectedSnack)));
  } catch (e) {
    if (!mounted) return;
    ScaffoldMessenger.of(
      context,
    ).showSnackBar(SnackBar(content: Text(_l10n.connectFirst)));
  } finally {
    if (mounted) setState(() => _loading = false);
  }
}

/// Envia um comando HTTP para o ESP32.
Future<bool> _send(String action) async {

```

```

if (!Esp32WifiService.instance.isConnected.value) {
    ScaffoldMessenger.of(
        context,
    ).showSnackBar(SnackBar(content: Text(_l10n.connectFirst)));
    return false;
}

try {
    await Esp32WifiService.instance.action(action);
    return true;
} catch (e) {
    if (!mounted) return false;
    ScaffoldMessenger.of(
        context,
    ).showSnackBar(SnackBar(content: Text(_l10n.actionFailed)));
    return false;
}
}

```

/// Marca todos os dedos como abertos na interface.

```

void _setOpenHand() {
    handClosed = false;
    peaceMode = false;
    indexClosed = false;
    middleClosed = false;
    ringPinkyClosed = false;
    thumbClosed = false;
}

```

/// Marca todos os dedos como fechados na interface.

```

void _setClosedHand() {
    handClosed = true;
    peaceMode = false;
    indexClosed = true;
    middleClosed = true;
    ringPinkyClosed = true;
    thumbClosed = true;
}

```

/// Atualiza os estados locais de acordo com o gesto executado.

```

void _applyGestureState(String action) {
    switch (action) {
        case 'open_hand':
            _setOpenHand();
            break;
        case 'close_hand':
            _setClosedHand();
    }
}

```

```
break;
case 'peace':
    handClosed = false;
    peaceMode = true;
    indexClosed = false;
    middleClosed = false;
    ringPinkyClosed = true;
    thumbClosed = true;
    break;
case 'two':
    handClosed = false;
    peaceMode = false;
    indexClosed = false;
    middleClosed = false;
    ringPinkyClosed = true;
    thumbClosed = true;
    break;
case 'point':
    handClosed = false;
    peaceMode = false;
    indexClosed = false;
    middleClosed = true;
    ringPinkyClosed = true;
    thumbClosed = true;
    break;
case 'three':
    handClosed = false;
    peaceMode = false;
    indexClosed = false;
    middleClosed = false;
    ringPinkyClosed = true;
    thumbClosed = false;
    break;
case 'four':
    handClosed = false;
    peaceMode = false;
    indexClosed = false;
    middleClosed = false;
    ringPinkyClosed = false;
    thumbClosed = true;
    break;
case 'thumbs_up':
    handClosed = false;
    peaceMode = false;
    indexClosed = true;
    middleClosed = true;
    ringPinkyClosed = true;
```

```

thumbClosed = false;
    break;
    case 'ok':
        handClosed = false;
        peaceMode = false;
        indexClosed = true;
        middleClosed = false;
        ringPinkyClosed = false;
        thumbClosed = false;
        break;
    case 'wave':
        _setOpenHand();
        break;
    case 'palm_up':
    case 'palm_down':
    case 'forearm_neutral':
        forearmKey = action;
        break;
    case 'demo':
        break;
}
}

```

/// Envia o gesto e atualiza a interface apenas quando o envio funciona.

```

Future<bool> _sendGestureAndUpdate(String action) async {
    final ok = await _send(action);
    if (!ok || !mounted) return false;
    setState(() => _applyGestureState(action));
    return true;
}

```

/// Executa um gesto completo, evitando conflitos com a demonstração.

```

Future<void> _runGesture(String action) async {
    if (!_demoRunning) return;

    if (action == 'demo') {
        await _runDemo();
        return;
    }

    if (action == 'wave') {
        final opened = await _sendGestureAndUpdate('open_hand');
        if (!opened) return;
        await Future<void>.delayed(const Duration(milliseconds: 180));
        await _sendGestureAndUpdate('wave');
        return;
    }
}

```

```

final sent = await _sendGestureAndUpdate(action);
if (!sent || !mounted) return;

final isHandGesture = ![
  'palm_up',
  'palm_down',
  'forearm_neutral',
].contains(action);

if (isHandGesture && forearmKey != 'forearm_neutral') {
  await Future<void>.delayed(const Duration(milliseconds: 120));
  await _send(forearmKey);
}
}

```

/// Executa a sequência automática de demonstração.

```

Future<void> _runDemo() async {
  if (_demoRunning) return;
  setState(() => _demoRunning = true);

```

```

final sequence = [
  'open_hand',
  'palm_up',
  'peace',
  'two',
  'palm_down',
  'point',
  'forearm_neutral',
  'three',
  'four',
  'palm_up',
  'thumbs_up',
  'palm_down',
  'ok',
  'forearm_neutral',

  'open_hand',
  'wave',
  'forearm_neutral',
  'open_hand',
];

```

```

for (final action in sequence) {
  if (!mounted) return;
  final ok = await _sendGestureAndUpdate(action);
  if (!ok) break;

```

```

await Future<void>.delayed(const Duration(milliseconds: 450));
}

if (mounted) {
  setState(() => _demoRunning = false);
}
}

/// Executa ações individuais dos dedos e atualiza só a parte afetada.
Future<void> _runFingerAction(String action) async {
  final ok = await _send(action);
  if (!ok || !mounted) return;

  setState(() {
    handClosed = false;
    peaceMode = false;

    switch (action) {
      case 'open_index':
        indexClosed = false;
        break;
      case 'close_index':
        indexClosed = true;
        break;
      case 'open_middle':
        middleClosed = false;
        break;
      case 'close_middle':
        middleClosed = true;
        break;
      case 'open_ringpinky':
        ringPinkyClosed = false;
        break;
      case 'close_ringpinky':
        ringPinkyClosed = true;
        break;
      case 'open_thumb':
        thumbClosed = false;
        break;
      case 'close_thumb':
        thumbClosed = true;
        break;
    }
  });

  if (forearmKey != 'forearm_neutral') {
    await Future<void>.delayed(const Duration(milliseconds: 120));
  }
}

```

```
await _send(forearmKey);
}
}
```

/// Alterna rapidamente entre abrir e fechar a mão.

```
Future<void> _toggleHand() async {
  await _runGesture(handClosed ? 'open_hand' : 'close_hand');
}
```

/// Liga o comando interpretado à ação real que deve ser enviada ao ESP32.

```
Future<void> _executeCommand(_ArmiacCommand command) async {
  switch (command) {
    case _ArmiacCommand.openHand:
      await _runGesture('open_hand');
      break;
    case _ArmiacCommand.closeHand:
      await _runGesture('close_hand');
      break;
    case _ArmiacCommand.peace:
      await _runGesture('peace');
      break;
    case _ArmiacCommand.two:
      await _runGesture('two');
      break;
    case _ArmiacCommand.point:
      await _runGesture('point');
      break;
    case _ArmiacCommand.three:
      await _runGesture('three');
      break;
    case _ArmiacCommand.four:
      await _runGesture('four');
      break;
    case _ArmiacCommand.thumbsUp:
      await _runGesture('thumbs_up');
      break;
    case _ArmiacCommand.ok:
      await _runGesture('ok');
      break;
    case _ArmiacCommand.wave:
      await _runGesture('wave');
      break;
    case _ArmiacCommand.palmUp:
      await _runGesture('palm_up');
      break;
    case _ArmiacCommand.palmDown:
      await _runGesture('palm_down');
```

```

break;
case _ArmiacCommand.forearmNeutral:
    await _runGesture('forearm_neutral');
    break;
case _ArmiacCommand.demo:
    await _runGesture('demo');
    break;
case _ArmiacCommand.openIndex:
    await _runFingerAction('open_index');
    break;
case _ArmiacCommand.closeIndex:
    await _runFingerAction('close_index');
    break;
case _ArmiacCommand.openMiddle:
    await _runFingerAction('open_middle');
    break;
case _ArmiacCommand.closeMiddle:
    await _runFingerAction('close_middle');
    break;
case _ArmiacCommand.openRingPinky:
    await _runFingerAction('open_ringpinky');
    break;
case _ArmiacCommand.closeRingPinky:
    await _runFingerAction('close_ringpinky');
    break;
case _ArmiacCommand.openThumb:
    await _runFingerAction('open_thumb');
    break;
case _ArmiacCommand.closeThumb:
    await _runFingerAction('close_thumb');
    break;
}
}

_ArmiacCommand? _commandFromAction(String action) {
switch (action) {
case 'open_hand':
    return _ArmiacCommand.openHand;
case 'close_hand':
    return _ArmiacCommand.closeHand;
case 'peace':
    return _ArmiacCommand.peace;
case 'two':
    return _ArmiacCommand.two;
case 'point':
    return _ArmiacCommand.point;
case 'three':

```

```

return _ArmiacCommand.three;
    case 'four':
        return _ArmiacCommand.four;
    case 'thumbs_up':
        return _ArmiacCommand.thumbsUp;
    case 'ok':
        return _ArmiacCommand.ok;
    case 'wave':
        return _ArmiacCommand.wave;
    case 'palm_up':
        return _ArmiacCommand.palmUp;
    case 'palm_down':
        return _ArmiacCommand.palmDown;
    case 'forearm_neutral':
        return _ArmiacCommand.forearmNeutral;
    case 'demo':
        return _ArmiacCommand.demo;
    case 'open_index':
        return _ArmiacCommand.openIndex;
    case 'close_index':
        return _ArmiacCommand.closeIndex;
    case 'open_middle':
        return _ArmiacCommand.openMiddle;
    case 'close_middle':
        return _ArmiacCommand.closeMiddle;
    case 'open_ringpinky':
        return _ArmiacCommand.openRingPinky;
    case 'close_ringpinky':
        return _ArmiacCommand.closeRingPinky;
    case 'open_thumb':
        return _ArmiacCommand.openThumb;
    case 'close_thumb':
        return _ArmiacCommand.closeThumb;
    }
    return null;
}

```

/// Escolhe a frase que será dita após cada comando reconhecido.

```

String _commandToSpeech(_ArmiacCommand command) {
    switch (command) {
        case _ArmiacCommand.openHand:
            return _l10n.speechFor('open_hand');
        case _ArmiacCommand.closeHand:
            return _l10n.speechFor('close_hand');
        case _ArmiacCommand.peace:
            return _l10n.speechFor('peace');
        case _ArmiacCommand.two:

```

```

return _l10n.speechFor('two');
    case _ArmiacCommand.point:
        return _l10n.speechFor('point');
    case _ArmiacCommand.three:
        return _l10n.speechFor('three');
    case _ArmiacCommand.four:
        return _l10n.speechFor('four');
    case _ArmiacCommand.thumbsUp:
        return _l10n.speechFor('thumbs_up');
    case _ArmiacCommand.ok:
        return _l10n.speechFor('ok');
    case _ArmiacCommand.wave:
        return _l10n.speechFor('wave');
    case _ArmiacCommand.palmUp:
        return _l10n.speechFor('palm_up');
    case _ArmiacCommand.palmDown:
        return _l10n.speechFor('palm_down');
    case _ArmiacCommand.forearmNeutral:
        return _l10n.speechFor('forearm_neutral');
    case _ArmiacCommand.demo:
        return _l10n.speechFor('demo');
    case _ArmiacCommand.openIndex:
        return _l10n.speechFor('open_index');
    case _ArmiacCommand.closeIndex:
        return _l10n.speechFor('close_index');
    case _ArmiacCommand.openMiddle:
        return _l10n.speechFor('open_middle');
    case _ArmiacCommand.closeMiddle:
        return _l10n.speechFor('close_middle');
    case _ArmiacCommand.openRingPinky:
        return _l10n.speechFor('open_ringpinky');
    case _ArmiacCommand.closeRingPinky:
        return _l10n.speechFor('close_ringpinky');
    case _ArmiacCommand.openThumb:
        return _l10n.speechFor('open_thumb');
    case _ArmiacCommand.closeThumb:
        return _l10n.speechFor('close_thumb');
    }
}

/// Interpreta o texto do utilizador e executa o comando correspondente.
Future<void> _runAiCommand(String text) async {
    setState(() {
        _aiFeedback = 'Texto ouvido: "$text"\nA interpretar o comando...';
    });

    final action = LocalCommandAiService.classify(text);

```

```

if (action == 'unknown') {
  setState(() {
    _aiFeedback = 'Não consegui entender: "$text"';
  });

  await _speakFast(_l10n.didNotUnderstand);
  return;
}

final command = _commandFromAction(action);

if (command == null) {
  setState(() {
    _aiFeedback = 'Comando inválido: $action';
  });
  return;
}

final spoken = _commandToSpeech(command);

setState(() {
  _aiFeedback =
    'Texto ouvido: "$text"\nComando identificado: $action\nResposta: $spoken';
});

await _speakFast(spoken);
await _executeCommand(command);
}

/// Escolhe o melhor idioma disponível para o reconhecimento de voz.
Future<String> _bestSpeechLocale() async {
  final preferred = appSettings.language == AppLanguage.portuguese
    ? 'pt_BR'
    : 'en_US';
  try {
    final locales = await _speech.locales();
    final exact = locales.where(
      (locale) => locale.localeId.toLowerCase() == preferred.toLowerCase(),
    );
    if (exact.isNotEmpty) return exact.first.localeId;

    final prefix = appSettings.language == AppLanguage.portuguese
      ? 'pt'
      : 'en';
    final sameLanguage = locales.where(
      (locale) => locale.localeId.toLowerCase().startsWith(prefix),
    );
  }
}

```

```

);
    if (sameLanguage.isNotEmpty) return sameLanguage.first.localeId;
  } catch (_) {}

}
return preferred;
}

/// Liga o microfone, recolhe a frase e envia o texto para interpretação.
Future<void> _listenVoiceCommand() async {
  if (_listening) {
    await _speech.stop();
    setState(() => _listening = false);
    return;
  }

  try {
    await _speech.cancel();
  } catch (_) {}
  _speech = stt.SpeechToText();

  final available = await _speech.initialize(
    onStatus: (status) {
      if ((status == 'done' || status == 'notListening') && mounted) {
        setState(() => _listening = false);
      }
    },
    onError: (error) {
      if (mounted) {
        setState(() {
          _listening = false;
          _aiFeedback = _speechErrorMessage(error.errorMsg);
        });
      }
    },
  );

  if (!available) {
    setState(() => _aiFeedback = _l10n.micUnavailable);
    return;
  }

  final localeId = await _bestSpeechLocale();

  setState(() {
    _listening = true;
    _lastVoiceText = "";
  });
}

```

```

_aiFeedback = '${_l10n.listening} ($localeId)';
});

try {
  await _speech.listen(
    localeId: localeId,
    onResult: (result) async {
      final words = result.recognizedWords;
      setState(() => _lastVoiceText = words);

      if (result.finalResult && words.trim().isEmpty) {
        await _speech.stop();
        if (mounted) setState(() => _listening = false);
        await _runAiCommand(words);
      }
    },
    listenFor: const Duration(seconds: 8),
    pauseFor: const Duration(seconds: 2),
    listenOptions: stt.SpeechListenOptions(
      onDevice: false,
      cancelOnError: true,
      partialResults: false,
    ),
  );
} catch (e) {
  if (mounted) {
    setState(() {
      _listening = false;
      _aiFeedback = _speechErrorMessage(e.toString());
    });
  }
}
}

```

/// Traduz erros do reconhecimento de voz para mensagens mais claras.

```

String _speechErrorMessage(String error) {
  final lower = error.toLowerCase();
  if (lower.contains('server') || lower.contains('disconnected')) {
    return _l10n.speechServerDisconnectedError;
  }
  if (lower.contains('network')) {
    return _l10n.speechNetworkError;
  }
  if (lower.contains('language') ||
    lower.contains('locale') ||
    lower.contains('not supported')) {
    return _l10n.speechLanguageError;
  }
}

```

```

}
return _l10n.micError(error);
}

/// Mostra uma lista de exemplos de comandos de voz.
void _openCommandHelp() {
  final l10n = _l10n;
  showModalBottomSheet<void>(
    context: context,
    isScrollControlled: true,
    builder: (context) => SafeArea(
      child: SingleChildScrollView(
        padding: const EdgeInsets.all(20),
        child: Column(
          mainAxisAlignment: MainAxisAlignment.min,
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Text(
              l10n.commandsHelp,
              style: const TextStyle(
                fontSize: 22,
                fontWeight: FontWeight.bold,
              ),
            ),
            const SizedBox(height: 8),
            Text(l10n.commandsHelpIntro),
            const SizedBox(height: 12),
            ...l10n.commandHelpItems.map(
              (item) => Padding(
                padding: const EdgeInsets.symmetric(vertical: 4),
                child: Text('• $item'),
              ),
            ),
          ],
        ),
      ),
    );
}

```

@override

```

Widget build(BuildContext context) {
  return AnimatedBuilder(
    animation: appSettings,
    builder: (context, _) {
      final l10n = _l10n;
      return ValueListenableBuilder<bool>(

```

```

valueListenable: Esp32WifiService.instance.isConnected,
builder: (context, connected, _) {
  return SingleChildScrollView(
    padding: const EdgeInsets.all(16),
    child: Column(
      mainAxisAlignment: MainAxisAlignment.center,
      children: [
        Text(
          connected ? I10n.connected : I10n.disconnected,
          style: const TextStyle(
            fontSize: 20,
            fontWeight: FontWeight.bold,
          ),
          textAlign: TextAlign.center,
        ),
        const SizedBox(height: 16),
        SizedBox(
          width: double.infinity,
          child: ElevatedButton.icon(
            onPressed: _loading ? null : _connect,
            icon: Icon(connected ? Icons.wifi : Icons.wifi_off),
            label: Text(
              _loading ? I10n.connecting : I10n.connectArmiac,
            ),
          ),
        ),
        const SizedBox(height: 12),
        SizedBox(
          width: double.infinity,
          child: ElevatedButton.icon(
            onPressed: connected ? _listenVoiceCommand : null,
            icon: Icon(_listening ? Icons.stop : Icons.mic),
            label: Text(
              _listening ? I10n.stopVoice : I10n.voiceControl,
            ),
          ),
        ),
        const SizedBox(height: 8),
        Wrap(
          alignment: WrapAlignment.center,
          spacing: 8,
          runSpacing: 4,
          children: [
            TextButton.icon(
              onPressed: _openTutorial,
              icon: const Icon(Icons.slideshow_outlined),
              label: Text(I10n.tutorial),
            ),
          ],
        ),
      ],
    ),
  ),
)

```

```

),
    TextButton.icon(
      onPressed: _openCommandHelp,
      icon: const Icon(Icons.help_outline),
      label: Text(l10n.help),
    ),
  ],
),
if (_lastVoiceText.isNotEmpty) ...[
  const SizedBox(height: 8),
  Text(
    '${l10n.voice}: $_lastVoiceText',
    textAlign: TextAlign.center,
  ),
],
if (_aiFeedback.isNotEmpty) ...[
  const SizedBox(height: 8),
  Text(_aiFeedback, textAlign: TextAlign.center),
],
const SizedBox(height: 20),
Center(
  child: Text(
    l10n.quickCommands,
    textAlign: TextAlign.center,
    style: const TextStyle(
      fontSize: 16,
      fontWeight: FontWeight.bold,
    ),
  ),
),
const SizedBox(height: 8),
Wrap(
  spacing: 8,
  runSpacing: 8,
  alignment: WrapAlignment.center,
  children: [
    ActionChip(
      label: Text(l10n.openCloseHand),
      onPressed: connected && !_demoRunning ? _toggleHand : null,
    ),
    ActionChip(
      label: Text(l10n.peace),
      onPressed: connected
        && !_demoRunning
        ? () => _runGesture('peace')
        : null,
    ),
  ],
),

```

```

ActionChip(
  label: Text(l10n.numberTwo),
  onPressed: connected
    && !_demoRunning
    ? () => _runGesture('two')
    : null,
),
ActionChip(
  label: Text(l10n.point),
  onPressed: connected
    && !_demoRunning
    ? () => _runGesture('point')
    : null,
),
ActionChip(
  label: Text(l10n.numberFour),
  onPressed: connected && !_demoRunning
    ? () => _runGesture('four')
    : null,
),
ActionChip(
  label: Text(l10n.thumbsUp),
  onPressed: connected && !_demoRunning
    ? () => _runGesture('thumbs_up')
    : null,
),
ActionChip(
  label: const Text('OK'),
  onPressed: connected && !_demoRunning ? () => _runGesture('ok') : null,
),
ActionChip(
  label: Text(l10n.wave),
  onPressed: connected && !_demoRunning ? () => _runGesture('wave') :
null,
),
ActionChip(
  label: Text(l10n.palmUpShort),
  onPressed: connected
    && !_demoRunning
    ? () => _runGesture('palm_up')
    : null,
),
ActionChip(
  label: Text(l10n.palmDownShort),
  onPressed: connected
    && !_demoRunning
    ? () => _runGesture('palm_down')

```

```

: null,
    ),
  ],
),
const SizedBox(height: 25),
SizedBox(
  width: double.infinity,
  child: ElevatedButton(
    onPressed: connected
      ? () {
        Navigator.push(
          context,
          MaterialPageRoute(
            builder: (_) => const ControlPage(),
          ),
        );
      }
    : null,
    child: Text(l10n.manualControl),
  ),
),
],
),
);
},
);
},
);
}
}
}

```

Diretório – proj_armiac/lib/home/tutorial_dialog.dart

```

import 'package:flutter/material.dart';

import '../app_language.dart';
import '../l10n.dart';

/// Janela com o tutorial rápido da aplicação.
class TutorialDialog extends StatefulWidget {
  const TutorialDialog({super.key});

  @override
  State<TutorialDialog> createState() => _TutorialDialogState();
}

```

```

class _TutorialDialogState extends State<TutorialDialog> {
  final PageController _controller = PageController();
  int _page = 0;

  L10n get _l10n => L10n(appSettings.language);

  @override
  /// Fecha corretamente o controlador das páginas.
  void dispose() {
    _controller.dispose();
    super.dispose();
  }

  /// Move o tutorial para a página pedida com animação.
  Future<void> _goTo(int page) async {
    await _controller.animateToPage(
      page,
      duration: const Duration(milliseconds: 250),
      curve: Curves.easeInOut,
    );
  }

  @override
  Widget build(BuildContext context) {
    final l10n = _l10n;
    final pages = [
      _TutorialCard(
        icon: Icons.power,
        title: l10n.tutorialStep1Title,
        body: l10n.tutorialStep1Body,
      ),
      _TutorialCard(
        icon: Icons.wifi,
        title: l10n.tutorialStep2Title,
        body: l10n.tutorialStep2Body,
        imageAsset: 'assets/tutorial/armiac_wifi.png',
      ),
    ];

    return Dialog(
      insetPadding: const EdgeInsets.symmetric(horizontal: 16, vertical: 20),
      child: ConstrainedBox(
        constraints: BoxConstraints(
          maxWidth: 380,
          maxHeight: MediaQuery.of(context).size.height * 0.88,
        ),
      ),
    );
  }
}

```

```

child: Padding(
  padding: const EdgeInsets.fromLTRB(20, 18, 20, 18),
  child: Column(
    mainAxisAlignment: MainAxisAlignment.min,
    children: [
      Row(
        children: [
          Expanded(
            child: Text(
              l10n.tutorialTitle,
              style: const TextStyle(
                fontSize: 22,
                fontWeight: FontWeight.bold,
              ),
            ),
          IconButton(
            onPressed: () => Navigator.of(context).pop(),
            icon: const Icon(Icons.close),
          ),
        ],
      ),
      const SizedBox(height: 10),
      SizedBox(
        height: 335,
        child: PageView.builder(
          controller: _controller,
          itemCount: pages.length,
          onPageChanged: (value) => setState(() => _page = value),
          itemBuilder: (context, index) => pages[index],
        ),
      ),
      const SizedBox(height: 12),
      Row(
        mainAxisAlignment: MainAxisAlignment.center,
        children: List.generate(
          pages.length,
          (index) => AnimatedContainer(
            duration: const Duration(milliseconds: 200),
            margin: const EdgeInsets.symmetric(horizontal: 4),
            width: _page == index ? 20 : 8,
            height: 8,
            decoration: BoxDecoration(
              color: _page == index
                ? Theme.of(context).colorScheme.primary
                : Colors.grey.shade400,
              borderRadius: BorderRadius.circular(99),
            ),
          ),
        ),
      ),
    ],
  ),
),

```

```

),
    ),
    ),
    ),
    const SizedBox(height: 14),
    Row(
      children: [
        if (_page > 0)
          TextButton(
            onPressed: () => _goTo(_page - 1),
            child: Text(l10n.previous),
          )
        else
          const SizedBox.shrink(),
        const Spacer(),
        ElevatedButton(
          onPressed: _page == pages.length - 1
            ? () => Navigator.of(context).pop()
            : () => _goTo(_page + 1),
          child: Text(
            _page == pages.length - 1 ? l10n.finish : l10n.next,
          ),
        ),
      ],
    ),
  ],
),
],
),
),
),
);
}
}

```

/// Cartão visual usado em cada passo do tutorial.

```

class _TutorialCard extends StatelessWidget {
  const _TutorialCard({
    required this.icon,
    required this.title,
    required this.body,
    this.imageAsset,
  });

```

```

  final IconData icon;
  final String title;
  final String body;
  final String? imageAsset;

```

@override

```
Widget build(BuildContext context) {  
  return Container(  
    width: double.infinity,  
    padding: const EdgeInsets.all(16),  
    decoration: BoxDecoration(  
      color: Theme.of(context).colorScheme.surfaceVariant,  
      borderRadius: BorderRadius.circular(20),  
    ),  
    child: SingleChildScrollView(  
      physics: const BouncingScrollPhysics(),  
      child: Column(  
        mainAxisAlignment: MainAxisAlignment.center,  
        children: [  
          if (imageAsset != null) ...[  
            ClipRRect(  
              borderRadius: BorderRadius.circular(16),  
              child: Container(  
                width: double.infinity,  
                height: 175,  
                color: Colors.white,  
                child: Image.asset(  
                  imageAsset!,  
                  fit: BoxFit.contain,  
                ),  
              ),  
            ),  
          ],  
          const SizedBox(height: 16),  
        ] else ...[  
          CircleAvatar(  
            radius: 38,  
            child: Icon(icon, size: 38),  
          ),  
          const SizedBox(height: 22),  
        ],  
        Text(  
          title,  
          textAlign: TextAlign.center,  
          style: const TextStyle(fontSize: 20, fontWeight: FontWeight.bold),  
        ),  
        const SizedBox(height: 10),  
        Text(  
          body,  
          textAlign: TextAlign.center,  
          style: const TextStyle(fontSize: 15, height: 1.3),  
        ),  
      ],  
    ),  
  ),  
],
```

```

),
),
);
}
}

```

Diretório – proj_armiac/lib/profile/profile_page.dart

```

import 'package:flutter/material.dart';

import '../app_language.dart';

/// Página simples reservada para dados de perfil.
class ProfilePage extends StatelessWidget {
  const ProfilePage({super.key});

  @override
  Widget build(BuildContext context) {
    final isPt = appSettings.language == AppLanguage.portuguese;
    return Center(
      child: Text(isPt ? 'Página Perfil' : 'Profile Page', style: const TextStyle(fontSize: 24)),
    );
  }
}

```

Diretório – proj_armiac/lib/services/ai_command_service.dart

```

import 'dart:convert';

import 'package:http/http.dart' as http;

import '../app_language.dart';

/// Serviço opcional para interpretar comandos com Gemini quando existir chave
/// configurada.
class AiCommandService {
  AiCommandService._();

  /// Instância única para evitar criar vários clientes do mesmo serviço.
  static final AiCommandService instance = AiCommandService._();
}

```

```

/// Chave lida em tempo de compilação através de --dart-define.
static const String geminiApiKey = String.fromEnvironment('GEMINI_API_KEY');

/// Lista de ações que a IA pode devolver e que o ESP32 sabe receber.
static const List<String> validCommands = [
  'open_hand',
  'close_hand',
  'peace',
  'point',
  'three',
  'thumbs_up',
  'ok',
  'wave',
  'palm_up',
  'palm_down',
  'forearm_neutral',
  'demo',
  'close_index',
  'open_index',
  'close_middle',
  'open_middle',
  'close_ringpinky',
  'open_ringpinky',
  'close_thumb',
  'open_thumb',
];

/// Indica se o serviço remoto pode ser usado.
bool get isConfigured => geminiApiKey.trim().isEmpty;

/// Envia a frase para o Gemini e tenta receber apenas uma ação válida.
Future<String?> understandWithGemini(String spokenText, AppLanguage language)
async {
  if (!isConfigured) return null;

  final uri = Uri.parse(
    'https://generativelanguage.googleapis.com/v1beta/models/gemini-1.5-
flash:generateContent?key=$geminiApiKey',
  );

  final languageName = language == AppLanguage.portuguese ? 'Portuguese' : 'English';
  final examples = language == AppLanguage.portuguese
    ? ""
    - abrir mão / abre a mão / resetar => open_hand

    - paz / vitória / número dois => peace

```

- apontar / número um => point
- número três / três dedos => three
- joinha / positivo => thumbs_up
- acenar / olá / adeus => wave
- palma para cima / supinação => palm_up
- palma para baixo / pronação => palm_down
- antebraço ao centro / posição neutra => forearm_neutral
- demonstração => demo

```

: ""

- open hand / reset / open all => open_hand
- close hand / fist / make a fist => close_hand
- peace / victory / number two => peace
- point / number one => point
- number three / three fingers => three
- thumbs up / like => thumbs_up
- wave / hello / bye => wave
- palm up / face up / supination => palm_up
- palm down / face down / pronation => palm_down
- neutral forearm / center forearm => forearm_neutral
- demo / demonstration => demo
",

```

final prompt = ""

You control a robotic hand called ARMIAC.

The app is currently set to \$languageName.

Only understand commands in \$languageName. If the user speaks another language, return UNKNOWN.

Return ONLY one command from this exact list:

`\${validCommands.join(', ')}`

Examples:

\$examples

If the phrase is unclear, return UNKNOWN.

User speech: "\$spokenText"

```

",

try {
  final response = await http
    .post(
      uri,
      headers: {'Content-Type': 'application/json'},
      body: jsonEncode({
        'contents': [
{
          'parts': [

```

```

        {'text': prompt}
    ]
}
],
'generationConfig': {
    'temperature': 0,
    'maxOutputTokens': 10,
}
}),
)
.timeout(const Duration(seconds: 6));

if (response.statusCode != 200) return null;

final data = jsonDecode(response.body) as Map<String, dynamic>;
final text = data['candidates']?[0]?['content']?['parts']?[0]?['text']
    ?.toString()
    .trim()
    .toLowerCase();

if (text == null || text == 'unknown') return null;
return validCommands.contains(text) ? text : null;
} catch (_) {
    return null;
}
}
}
}

```

Diretório – proj_armiac/lib/services/esp32_wifi_service.dart

```

import 'dart:convert';

import 'package:flutter/foundation.dart';
import 'package:http/http.dart' as http;

/// Serviço responsável pela comunicação HTTP com o ESP32.
class Esp32WifiService {
    Esp32WifiService._();
    /// Instância única usada por todas as páginas.
    static final Esp32WifiService instance = Esp32WifiService._();

    /// Endereço padrão do ESP32 quando está a funcionar como ponto de acesso Wi-Fi.
    static const String baseUrl = 'http://192.168.4.1';
    final ValueNotifier<bool> isConnected = ValueNotifier<bool>(false);

    /// Verifica se o ESP32 responde e guarda o estado da ligação.
    Future<void> connect() async {
        final ok = await ping();
    }
}

```

```

    isConnected.value = ok;
    if (!ok) throw Exception('ARMIAC_WIFI_NOT_CONNECTED');
  }

  /// Faz um pedido simples para confirmar que o servidor está ativo.
  Future<bool> ping() async {
    try {
      final r = await http.get(Uri.parse('$baseUrl/ping')).timeout(const Duration(seconds: 2));
      return r.statusCode == 200;
    } catch (_) {
      return false;
    }
  }

  /// Lê o estado atual devolvido pelo ESP32.
  Future<Map<String, dynamic>?> status() async {
    try {
      final r = await http.get(Uri.parse('$baseUrl/status')).timeout(const Duration(seconds:
2));
      if (r.statusCode != 200) return null;
      return jsonDecode(r.body) as Map<String, dynamic>;
    } catch (_) {
      return null;
    }
  }

  /// Envia uma ação para o endpoint /action.
  Future<void> action(String a) async {
    final uri = Uri.parse('$baseUrl/action').replace(queryParameters: {'a': a});
    final r = await http.get(uri).timeout(const Duration(seconds: 3));
    if (r.statusCode != 200) {
      throw Exception('ARMIAC_ACTION_FAILED');
    }
  }
}

```

Diretório – proj_armiac/lib/services/local_command_ai_service.dart

```

import 'dart:math';

/// Interpretador local de comandos de voz, sem depender de internet.
class LocalCommandAiService {
  /// Frases muito específicas que identificam diretamente um comando.
  static const Map<String, List<String>> strongPhrases = {
    'open_hand': [
      'abrir mao',
      'abre mao',
      'abra a mao'
    ]
  }
}

```

```
'abrir a mao',
'abrir os dedos',
'abre os dedos',
'abrir todos os dedos',
'abre todos os dedos',
'deixa a mao aberta',
'solta a mao',
'liberta a mao',
'abre completamente',
'abre totalmente a mao',
'estende a mao',
'deixa os dedos abertos',
'estica os dedos',
'mao aberta',
'open hand',
'open the hand',
'open fingers',
'open all fingers',
'release hand',
'stretch fingers',
'extend hand',
],
'close_hand': [
'fechar mao',
'fecha mao',
'fecha a mao',
'fechar a mao',
'feche a mao',
'feche os dedos',
'fecha os dedos',
'fechar os dedos',
'fechar todos os dedos',
'fecha todos os dedos',
'fazer punho',
'faz punho',
'fecha tudo',
'fecha os dedos todos',
'fecha a mao toda',
'junta os dedos',
'contraí a mao',
'aperta a mao',
'mao fechada',
'punho fechado',
'close hand',
'close the hand',
'make a fist',
'closed fist',
```

```
'squeeze hand',
'close all fingers',
'close the fingers',
],
'two': [
'numero dois',
'numero 2',
'dois',
'dois dedos',
'mostra dois',
'mostra dois dedos',
'faz dois',
'faz o dois',
'levanta dois dedos',
'number two',
'two',
'two fingers',
'show two',
],
'peace': [
'sinal de paz',
'faz paz',
'paz',
'vitoria',
'faz o v',
'sinal v',
'gesto v',
'faz vitoria',
'peace sign',
'victory sign',
'make a v',
'v sign',
],
'point': [
'apontar',
'aponta',
'aponta com o dedo',
'dedo indicador',
'deixa so o indicador',
'so o indicador',
'aponta em frente',
'aponta para frente',
'indicador aberto',
'numero um',
'faz numero um',
'point',
'point finger',
```

```
'index finger',
'only index',
'point forward',
'number one',
],
'three': [
'tres dedos',
'numero tres',
'numero 3',
'faz tres',
'faz o tres',
'mostra tres dedos',
'levanta tres dedos',
'abre tres dedos',
'mostra o tres',
'three fingers',
'number three',
'show three',
],
'four': [
'quatro dedos',
'numero quatro',
'numero 4',
'quatro',
'faz quatro',
'faz o quatro',
'mostra quatro',
'mostra quatro dedos',
'levanta quatro dedos',
'four fingers',
'number four',
'show four',
],
'thumbs_up': [
'joinha',
'joia',
'faz joinha',
'faz positivo',
'sinal positivo',
'polegar para cima',
'dedao para cima',
'fixe',
'gosto',
'like',
'thumbs up',
'positive sign',
'thumb up',
```

```
],
'ok': [
    'ok',
    'okay',
    'faz ok',
    'sinal de ok',
    'esta bem',
    'tudo bem',
    'certo',
    'perfeito',
    'confirmar',
    'confirma',
    'gesto ok',
    'okay sign',
],
'wave': [
    'acenar',
    'acena',
    'dizer ola',
    'diz ola',
    'dizer adeus',
    'diz adeus',
    'cumprimentar',
    'manda ola',
    'diz xau',
    'ola',
    'adeus',
    'cumprimenta',
    'wave',
    'say hello',
    'say goodbye',
    'greet',
],
'palm_up': [
    'palma para cima',
    'palma cima',
    'mao para cima',
    'vira a palma para cima',
    'vira para cima',
    'roda para cima',
    'supinacao',
    'palm up',
    'rotate up',
    'supination',
],
'palm_down': [
    'palma para baixo',
```

```
'palma baixo',
'mao para baixo',
'vira a palma para baixo',
'vira para baixo',
'roda para baixo',
'pronacao',
'palm down',
'rotate down',
'pronation',
],
'forearm_neutral': [
  'antebraco neutro',
  'posicao neutra',
  'posicao normal',
  'volta ao normal',
  'voltar ao normal',
  'fica normal',
  'volta ao centro',
  'antebraco ao centro',
  'centro',
  'neutral position',
  'normal position',
  'center forearm',
  'return to center',
],
'demo': [
  'demo',
  'demonstracao',
  'faz demonstracao',
  'modo demonstracao',
  'mostra tudo',
  'testa tudo',
  'testar movimentos',
  'faz o teste',
  'modo teste',
  'demo mode',
  'show everything',
  'test movements',
  'run demo',
],
'open_index': [
  'abre o indicador',
  'abrir o indicador',
  'abre dedo indicador',
  'abre o dedo indicador',
  'abrir dedo indicador',
  'indicador aberto',
```

```
'open index',
'open index finger',
],
'close_index': [
'fecha o indicador',
'fechar o indicador',
'feche o indicador',
'fecha dedo indicador',
'fecha o dedo indicador',
'fechar dedo indicador',
'indicador fechado',
'close index',
'close index finger',
],
'open_middle': [
'abre o dedo medio',
'abrir o dedo medio',
'abre o medio',
'abre dedo medio',
'dedo medio aberto',
'open middle finger',
'open middle',
],
'close_middle': [
'fecha o dedo medio',
'fechar o dedo medio',
'feche o dedo medio',
'fecha o medio',
'fecha dedo medio',
'dedo medio fechado',
'close middle finger',
'close middle',
],
'open_ringpinky': [
'abre o anelar e o mindinho',
'abrir o anelar e o mindinho',
'abre anelar e mindinho',
'abre o mindinho e o anelar',
'anelar e mindinho abertos',
'open ring and pinky',
'open ring pinky',
],
'close_ringpinky': [
'fecha o anelar e o mindinho',
'fechar o anelar e o mindinho',
'feche o anelar e o mindinho',
'fecha anelar e mindinho',
```

```

        'fecha o mindinho e o anelar',
        'anelar e mindinho fechados',
        'close ring and pinky',
        'close ring pinky',
    ],
    'open_thumb': [
        'abre o polegar',
        'abrir o polegar',
        'abre o dedao',
        'abre dedao',
        'polegar aberto',
        'dedao aberto',
        'open thumb',
    ],
    'close_thumb': [
        'fecha o polegar',
        'fechar o polegar',
        'feche o polegar',
        'fecha o dedao',
        'fecha dedao',
        'polegar fechado',
        'dedao fechado',
        'close thumb',
    ],
};

```

/// Palavras e expressões que ajudam a pontuar cada comando possível.

```
static const Map<String, List<String>> keywords = {
```

```

    'open_hand': [
        'abrir',
        'abre',
        'aberta',
        'aberto',
        'estica',
        'esticar',
        'solta',
        'liberta',
        'estende',
        'dedos',
        'mao',
    ],
    'close_hand': [
        'fechar',
        'fecha',
        'fechada',
        'fechado',
        'punho',
    ],
};

```

```
'aperta',
'apertar',
'contrai',
'dedos',
'mao',
],
'two': [
'dois',
'2',
'dedos',
'numero',
'two',
],
'peace': [
'paz',
'vitoria',
'v',
'peace',
'victory',
],
'point': [
'apontar',
'aponta',
'indicador',
'frente',
'dedo',
'point',
'index',
'um',
],
'three': [
'tres',
'3',
'mostra',
'dedos',
'numero',
'three',
],
'four': [
'quatro',
'4',
'mostra',
'dedos',
'numero',
'four',
],
'thumbs_up': [
```

```
'joinha',
'joia',
'positivo',
'like',
'polegar',
'dedao',
'thumb',
'cima',
'fixe',
],
'ok': [
'ok',
'okay',
'certo',
'perfeito',
'bem',
'confirmar',
'confirma',
],
'wave': [
'acenar',
'acena',
'ola',
'adeus',
'cumprimenta',
'cumprimentar',
'wave',
'hello',
'goodbye',
],
'palm_up': [
'palma',
'mao',
'cima',
'roda',
'supinacao',
'up',
],
'palm_down': [
'palma',
'mao',
'baixo',
'roda',
'pronacao',
'down',
],
'forearm_neutral': [
```

```

    'neutro',
    'neutra',
    'normal',
    'centro',
    'centraliza',
    'volta',
    'voltar',
    'posicao',
    'antebraco',
    'neutral',
],
'demo': [
    'demo',
    'demonstracao',
    'mostra',
    'testa',
    'testar',
    'movimentos',
    'teste',
],
'open_index': ['abrir', 'abre', 'indicador', 'index', 'open'],
'close_index': ['fechar', 'fecha', 'indicador', 'index', 'close'],
'open_middle': ['abrir', 'abre', 'medio', 'middle', 'open'],
'close_middle': ['fechar', 'fecha', 'medio', 'middle', 'close'],
'open_ringpinky': [
    'abrir',
    'abre',
    'anelar',
    'mindinho',
    'pinky',
    'ring',
    'open',
],
'close_ringpinky': [
    'fechar',
    'fecha',
    'anelar',
    'mindinho',
    'pinky',
    'ring',
    'close',
],
'open_thumb': ['abrir', 'abre', 'polegar', 'dedao', 'thumb', 'open'],
'close_thumb': ['fechar', 'fecha', 'polegar', 'dedao', 'thumb', 'close'],
};

```

/// Palavras obrigatórias para evitar escolher comandos errados por coincidência.

```

static const Map<String, List<String>> requiredAny = {
  'open_hand': [
    'abrir',
    'abre',
    'aberta',
    'aberto',
    'estica',
    'esticar',
    'solta',
    'liberta',
    'estende',
  ],
  'close_hand': [
    'fechar',
    'fecha',
    'fechada',
    'fechado',
    'punho',
    'aperta',
    'apertar',
    'contraí',
  ],
  'two': ['dois', '2', 'two'],
  'peace': ['paz', 'vitoria', 'v', 'peace', 'victory'],
  'point': ['apontar', 'aponta', 'indicador', 'point', 'index', 'um'],
  'three': ['tres', '3', 'three'],
  'four': ['quatro', '4', 'four'],
  'thumbs_up': ['joinha', 'joia', 'positivo', 'like', 'polegar', 'dedao', 'thumb', 'fixe'],
  'ok': ['ok', 'okay', 'certo', 'perfeito', 'confirmar', 'confirma'],
  'wave': ['acenar', 'acena', 'ola', 'adeus', 'cumprimenta', 'wave', 'hello', 'goodbye'],
  'palm_up': ['cima', 'roda', 'supinacao', 'up'],
  'palm_down': ['baixo', 'roda', 'pronacao', 'down'],
  'forearm_neutral': ['neutro', 'neutra', 'normal', 'centro', 'neutral'],
  'demo': ['demo', 'demonstracao', 'mostra', 'testa', 'testar', 'teste'],
  'open_index': ['indicador', 'index'],
  'close_index': ['indicador', 'index'],
  'open_middle': ['medio', 'middle'],
  'close_middle': ['medio', 'middle'],
  'open_ringpinky': ['anelar', 'mindinho', 'pinky', 'ring'],
  'close_ringpinky': ['anelar', 'mindinho', 'pinky', 'ring'],
  'open_thumb': ['polegar', 'dedao', 'thumb'],
  'close_thumb': ['polegar', 'dedao', 'thumb'],
};

```

/// Palavras comuns que não ajudam a decidir o comando.

```

static const Set<String> fillerWords = {
  'por',

```

```
'favor',
'podes',
'pode',
'podia',
'consegue',
'consegues',
'quero',
'queria',
'faz',
'fazer',
'mete',
'coloca',
'deixa',
'bota',
'agora',
'ai',
'tipo',
'assim',
'me',
'para',
'se',
'la',
'o',
'a',
'os',
'as',
'ao',
'um',
'uma',
'e',
'pff',
'porfavor',
'please',
'can',
'you',
'make',
'do',
'the',
'an',
};
```

/// Correções simples para normalizar frases antes da comparação.

```
static const Map<String, String> replacements = {
    'maun': 'mao',
    'mão': 'mao',
    'maos': 'mao',
    'mãos': 'mao',

```

```

'fexar': 'fechar',
'fexa': 'fecha',
'feixa': 'fecha',
'feixar': 'fechar',
'jóinha': 'joinha',
'joínha': 'joinha',
'join': 'joinha',
'jóia': 'joia',
'dedão': 'dedao',
'médio': 'medio',
'meio': 'medio',
'indicado': 'indicador',
'indica dor': 'indicador',
'vitória': 'vitoria',
'três': 'tres',
'tree': 'three',
'palm': 'palma',
'pra': 'para',
'xau': 'adeus',
'chau': 'adeus',
'tá': 'esta',
'ta': 'esta',
'num': 'numero',
};

```

/// Recebe uma frase e devolve a ação mais provável.

```

static String classify(String text) {
    final cleanText = normalize(text);

    if (cleanText.isEmpty) {
        return 'unknown';
    }

    final scores = <String, double>{};

    for (final command in strongPhrases.keys) {
        scores[command] = _scoreCommand(cleanText, command);
    }

    final ranked = scores.entries.toList()
        ..sort((a, b) => b.value.compareTo(a.value));

    final best = ranked[0];
    final second =
        ranked.length > 1 ? ranked[1] : const MapEntry('unknown', 0.0);

    if (best.value < 2.2) {

```

```

    return 'unknown';
}

if (second.value > 0 && best.value - second.value < 0.45) {
    return 'unknown';
}

return best.key;
}

/// Limpa acentos, pontuação e espaços para facilitar a comparação.
static String normalize(String text) {
    var value = text.toLowerCase().trim();

    value = _removeAccents(value);

    for (final entry in replacements.entries) {
        final wrong = _removeAccents(entry.key.toLowerCase());
        final right = _removeAccents(entry.value.toLowerCase());
        value = value.replaceAll(RegExp("\\b${RegExp.escape(wrong)}\\b"), right);
    }

    value = value.replaceAll(RegExp(r'^a-z0-9\\s'), ' ');
    value = value.replaceAll(RegExp(r'\\s+'), ' ').trim();

    return value;
}

```

```

/// Divide a frase em palavras úteis para o cálculo de pontuação.
static List<String> _tokens(String text) {
    return normalize(text)
        .split(' ')
        .where((word) => word.isNotEmpty && !fillerWords.contains(word))
        .toList();
}

```

```

/// Calcula a pontuação de um comando com base em frases, palavras e semelhança.
static double _scoreCommand(String cleanText, String command) {
    final inputTokens = _tokens(cleanText);
    var score = 0.0;

    final phrases = strongPhrases[command] ?? [];
    final commandKeywords = keywords[command] ?? [];

    for (final phrase in phrases) {
        final cleanPhrase = normalize(phrase);

```

```

if (cleanText.contains(cleanPhrase)) {
    score += 3.0;
}

final phraseScore = _similarity(cleanText, cleanPhrase);

if (phraseScore >= 0.75) {
    score += phraseScore * 2.5;
}

final overlap = _tokenOverlapScore(inputTokens, _tokens(cleanPhrase));
score += overlap * 2.0;
}

for (final keyword in commandKeywords) {
    final cleanKeyword = normalize(keyword);

    for (final word in inputTokens) {
        final sim = _similarity(word, cleanKeyword);

        if (word == cleanKeyword) {
            score += 0.9;
        } else if (sim >= 0.82) {
            score += 0.6;
        } else if (sim >= 0.72) {
            score += 0.3;
        }
    }
}

if (!_containsRequiredWord(inputTokens, requiredAny[command] ?? [])) {
    score *= 0.35;
}

final joined = inputTokens.join(' ');
final fingerWords = [
    'indicador',
    'medio',
    'anelar',
    'mindinho',
    'polegar',
    'dedao',
    'index',
    'middle',
    'ring',
    'pinky',
    'thumb',

```

```

];

if (command == 'open_hand' &&
    _hasAny(joined, ['fechar', 'fecha', 'punho', 'fechada'])) {
    score *= 0.25;
}

if (command == 'close_hand' &&
    _hasAny(joined, ['abrir', 'abre', 'aberta', 'estica'])) {
    score *= 0.25;
}

if ((command == 'open_hand' || command == 'close_hand') &&
    _hasAny(joined, fingerWords) &&
    !_hasAny(joined, ['todos', 'tudo', 'mao', 'dedos'])) {
    score *= 0.25;
}

if (command == 'palm_up' && joined.contains('baixo')) {
    score *= 0.1;
}

if (command == 'palm_down' && joined.contains('cima')) {
    score *= 0.1;
}

if (command == 'peace' && _hasAny(joined, ['dois', '2', 'tres', '3', 'quatro', '4'])) {
    score *= 0.3;
}

if (command == 'two' && _hasAny(joined, ['paz', 'vitoria', 'tres', '3', 'quatro', '4'])) {
    score *= 0.3;
}

if (command == 'three' &&
    _hasAny(joined, ['dois', '2', 'paz', 'vitoria', 'quatro', '4'])) {
    score *= 0.3;
}

if (command == 'four' && _hasAny(joined, ['dois', '2', 'tres', '3', 'paz', 'vitoria'])) {
    score *= 0.3;
}

if (command == 'thumbs_up' && joined.contains('palma')) {
    score *= 0.35;
}

```

```

if (command == 'palm_up' && _hasAny(joined, ['joinha', 'polegar', 'thumb'])) {
    score *= 0.35;
}

if (command.startsWith('open_') &&
    !command.contains('hand') &&
    _hasAny(joined, ['fechar', 'fecha', 'feche'])) {
    score *= 0.3;
}

if (command.startsWith('close_') &&
    !command.contains('hand') &&
    _hasAny(joined, ['abrir', 'abre'])) {
    score *= 0.3;
}

return score;
}

/// Confirma se a frase tem pelo menos uma palavra obrigatória para o comando.
static bool _containsRequiredWord(
    List<String> inputTokens,
    List<String> requiredWords,
) {
    if (requiredWords.isEmpty) return true;

    for (final required in requiredWords) {
        final cleanRequired = normalize(required);

        for (final word in inputTokens) {
            if (word == cleanRequired || _similarity(word, cleanRequired) >= 0.82) {
                return true;
            }
        }
    }

    return false;
}

/// Verifica se alguma expressão de uma lista aparece no texto.
static bool _hasAny(String text, List<String> words) {
    return words.any(text.contains);
}

/// Mede quantas palavras importantes coincidem entre duas expressões.
static double _tokenOverlapScore(
    List<String> inputTokens,

```

```

List<String> patternTokens,
) {
    if (patternTokens.isEmpty) return 0.0;

    var score = 0.0;

    for (final pattern in patternTokens) {
        var best = 0.0;

        for (final token in inputTokens) {
            best = max(best, _similarity(token, pattern));
        }

        if (best >= 0.82) {
            score += 1.0;
        } else if (best >= 0.68) {
            score += 0.6;
        }
    }

    return score / patternTokens.length;
}

/// Calcula uma semelhança simples entre duas frases.
static double _similarity(String a, String b) {
    if (a == b) return 1.0;
    if (a.isEmpty || b.isEmpty) return 0.0;

    final matrix = List.generate(
        a.length + 1,
        (_) => List<int>.filled(b.length + 1, 0),
    );

    for (var i = 0; i <= a.length; i++) {
        matrix[i][0] = i;
    }

    for (var j = 0; j <= b.length; j++) {
        matrix[0][j] = j;
    }

    for (var i = 1; i <= a.length; i++) {
        for (var j = 1; j <= b.length; j++) {
            final cost = a[i - 1] == b[j - 1] ? 0 : 1;

            matrix[i][j] = min(
                min(

```

```

        matrix[i - 1][j] + 1,
        matrix[i][j - 1] + 1,
    ),
    matrix[i - 1][j - 1] + cost,
);
}
}

final distance = matrix[a.length][b.length];
final maxLength = max(a.length, b.length);

return 1.0 - distance / maxLength;
}

/// Remove acentos para comparar português e inglês de forma mais estável.
static String _removeAccents(String text) {
  const from = 'áàãâäëèêëîïĩíóòõôóúùûüçñ';
  const to = 'aaaaaeeeeeiioooooouuuuchn';

  var result = text;

  for (var i = 0; i < from.length; i++) {
    result = result.replaceAll(from[i], to[i]);
  }

  return result;
}
}

```

Diretório – proj_armiac/lib/settings/settings_page.dart

```

import 'package:flutter/material.dart';
import 'package:package_info_plus/package_info_plus.dart';

import '../app_language.dart';
import '../l10n.dart';
import 'terms_page.dart';

class SettingsPage extends StatefulWidget {
  const SettingsPage({super.key});

  @override
  State<SettingsPage> createState() => _SettingsPageState();
}

/// Carrega dados da aplicação e apresenta opções de idioma e ajuda

```

```

class _SettingsPageState extends State<SettingsPage> {
  String appVersion = "";

  @override
  /// Carrega a versão assim que a página é aberta.
  void initState() {
    super.initState();
    _loadVersion();
  }

  /// Obtém a versão definida no pacote da aplicação.
  Future<void> _loadVersion() async {
    final info = await PackageInfo.fromPlatform();
    if (!mounted) return;
    setState(() => appVersion = info.version);
  }

  /// Cria cartões informativos reutilizáveis.
  Widget _infoTile({
    required IconData icon,
    required String title,
    required String subtitle,
  }) {
    return Card(
      child: ListTile(
        leading: Icon(icon),
        title: Text(title, style: const TextStyle(fontWeight: FontWeight.bold)),
        subtitle: Text(subtitle),
      ),
    );
  }

  @override
  Widget build(BuildContext context) {
    return AnimatedBuilder(
      animation: appSettings,
      builder: (context, _) {
        final l10n = L10n(appSettings.language);

        return ListView(
          padding: const EdgeInsets.all(16),
          children: [
            Text(l10n.settings, style: const TextStyle(fontSize: 24, fontWeight:
FontWeight.bold)),
            const SizedBox(height: 16),
            Card(
              child: ListTile(

```

```

        leading: ClipRRect(
          borderRadius: BorderRadius.circular(10),
          child: Image.asset('assets/images/Armiac_LOGO.png', width: 48, height: 48, fit:
BoxFit.contain),
        ),
        title: Text(l10n.appName, style: const TextStyle(fontWeight: FontWeight.bold)),
        subtitle: Text(l10n.appSubtitle),
      ),
    ),
    Card(
      child: Padding(
        padding: const EdgeInsets.all(16),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Text(l10n.languageLabel, style: const TextStyle(fontWeight: FontWeight.bold)),
            const SizedBox(height: 12),
            SegmentedButton<AppLanguage>(
              segments: [
                ButtonSegment(
                  value: AppLanguage.portuguese,
                  label: Text(l10n.portuguese),
                  icon: const Text('PT'),
                ),
                ButtonSegment(
                  value: AppLanguage.english,
                  label: Text(l10n.english),
                  icon: const Text('GB'),
                ),
              ],
              selected: {appSettings.language},
              onSelectionChanged: (selection) {
                appSettings.setLanguage(selection.first);
              },
            ),
            const SizedBox(height: 10),
            Text(l10n.languageHelp, style: const TextStyle(color: Colors.grey)),
          ],
        ),
      ),
    ),
    _infoTile(
      icon: Icons.mic,
      title: l10n.microphonePermission,
      subtitle: l10n.microphonePermissionBody,
    ),
    _infoTile(

```

```

        icon: Icons.wifi,
        title: l10n.armiacConnection,
        subtitle: l10n.armiacConnectionBody,
      ),
      const SizedBox(height: 12),
      Text(
        '${l10n.appVersion}: $appVersion',
        style: const TextStyle(fontSize: 16, color: Colors.grey),
      ),
      const SizedBox(height: 24),
      Center(
        child: TextButton.icon(
          onPressed: () => Navigator.push(
            context,
            MaterialPageRoute(builder: (_) => const TermsPage()),
          ),
          icon: const Icon(Icons.open_in_new),
          label: Text(l10n.serviceTerms),
        ),
      ),
    ],
  );
}
);
}
}

```

Diretório – proj_armiac/lib/settings/terms_page.dart

```

import 'package:flutter/material.dart';

import '../app_language.dart';
import '../l10n.dart';

/// Página com os termos de serviço.
class TermsPage extends StatefulWidget {
  const TermsPage({super.key});

  @override
  State<TermsPage> createState() => _TermsPageState();
}

/// Controla as animações de entrada do texto dos termos.
class _TermsPageState extends State<TermsPage> with SingleTickerProviderStateMixin {
  late final AnimationController _controller;
  late final Animation<double> _fade;
  late final Animation<Offset> _slide;

```

```

@override
/// Prepara e inicia a animação de entrada.
void initState() {
  super.initState();
  _controller = AnimationController(vsync: this, duration: const Duration(milliseconds:
450));
  _fade = CurvedAnimation(parent: _controller, curve: Curves.easeOut);
  _slide = Tween<Offset>(begin: const Offset(0, 0.04), end: Offset.zero).animate(_fade);
  _controller.forward();
}

```

```

@override
/// Liberta o controlador da animação.
void dispose() {
  _controller.dispose();
  super.dispose();
}

```

```

@override
Widget build(BuildContext context) {
  return AnimatedBuilder(
    animation: appSettings,
    builder: (context, _) {
      final l10n = L10n(appSettings.language);
      final cs = Theme.of(context).colorScheme;

      return Scaffold(
        appBar: AppBar(
          title: Text(l10n.termsTitle),
          backgroundColor: cs.inversePrimary,
        ),
        body: FadeTransition(
          opacity: _fade,
          child: SlideTransition(
position: _slide,
            child: ListView(
              padding: const EdgeInsets.fromLTRB(16, 16, 16, 24),
              children: [
                Container(
                  padding: const EdgeInsets.all(16),
                  decoration: BoxDecoration(
                    borderRadius: BorderRadius.circular(18),
                    gradient: LinearGradient(
                      colors: [
                        cs.primary.withOpacity(0.12),
                        cs.secondary.withOpacity(0.10),
                      ],

```

```

        begin: Alignment.topLeft,
        end: Alignment.bottomRight,
    ),
    border: Border.all(color: cs.primary.withOpacity(0.18)),
),
child: Column(
    crossAxisAlignment: CrossAxisAlignment.start,
    children: [
        Text(
            l10n.termsHeader,
            style: const TextStyle(fontSize: 18, fontWeight: FontWeight.w900),
        ),
        const SizedBox(height: 8),
        Row(
            children: [
                Icon(Icons.event, size: 18, color: cs.primary),
                const SizedBox(width: 8),
                Text(
                    l10n.lastUpdate,
                    style: const TextStyle(fontSize: 13.5, fontWeight: FontWeight.w600),
                ),
            ],
        ),
    ],
),
const SizedBox(height: 18),
...l10n.termsParagraphs.map(
    (text) => Padding(
        padding: const EdgeInsets.only(bottom: 12),
        child: Text(text, style: const TextStyle(fontSize: 15.5, height: 1.45)),
    ),
),
],
),
),
),
);
},
);
}
}

```

pubspec.yaml Code:

Directory - proj_armiac/pubspec.yaml

name: proj_armiac
description: "Desenvolvido por Quévin Tavares"

publish_to: 'none'

version: 1.1.7+9

environment:
 sdk: ">=3.10.0 <4.0.0"

dependencies:
 flutter:
 sdk: flutter
 cupertino_icons: ^1.0.8
 package_info_plus: ^8.0.2
 http: ^1.2.2
 speech_to_text: ^7.3.0
 flutter_tts: ^4.2.0
 shared_preferences: ^2.3.2

dev_dependencies:
 flutter_test:
 sdk: flutter
 flutter_lints: ^6.0.0

flutter:
 uses-material-design: true
 assets:
 - assets/images/Armiac_LOGO.png
- assets/tutorial

7.2 - ARMIAC Application APK

Application APK:

https://drive.google.com/file/d/1pG642exOyXLrIG7X_opwh_tH_ayCOEfX/view?usp=sharing

7.3 - YouTube Video Links

Examples of base projects

- Project 1: <https://youtu.be/L2ddw3MQI9g?si=44-PRt1J8Qk6TCc3>
- Project 2: https://www.tiktok.com/@pathofseb/video/7523183373288869133?is_from_webapp=1&sender_device=pc

3D printing

<https://youtube.com/shorts/XccsXvw3TjE?si=Ub7Kg75nmciBa69N>

First servomotor test

<https://youtube.com/shorts/2wemvdqr7-U?si=sgkJaba-hj45bRa1>

Demonstration of finger operation

https://youtube.com/shorts/j9Mxo_9-ovw?si=rJqrfvmzXTgFEHxD

Arm demonstration

<https://youtube.com/shorts/-pQoMxjrOXw?si=P2gNtikCYM4VFR2T>

Demonstration of the unfinished project

- Video 1: https://youtu.be/dLHhLJuH_1o?si=sjqv0j5o4DHVVhbd
- Video 2: <https://youtube.com/shorts/C5wIR47ePRw?si=5v5-RfpaooyCxAZo>

Demonstration of the project with battery vs. power supply

<https://youtu.be/RV1m9Q8S5C0?si=15LIKdADrWPj9K7f>